

# **On How Transformers Recognise Quantifier Depth in First-Order Logic**

A dissertation submitted to The University of Manchester for the degree of  
**Master of Science in Computational and Corpus Linguistics**  
in the Faculty of Linguistics and English Language

**Year of submission**  
2025

**Course Code**  
LELA71000

**Student ID**  
11583741

**Supervisor**  
Dmitry Nikolaev

School of Arts, Languages and Cultures

# Contents

|                                        |           |
|----------------------------------------|-----------|
| <b>Contents</b>                        | <b>2</b>  |
| <b>List of Figures</b>                 | <b>4</b>  |
| <b>List of Tables</b>                  | <b>5</b>  |
| <b>Terms and Abbreviations</b>         | <b>6</b>  |
| <b>Abstract</b>                        | <b>8</b>  |
| <b>Declaration</b>                     | <b>9</b>  |
| <b>Intellectual Property Statement</b> | <b>9</b>  |
| <b>1 Introduction</b>                  | <b>10</b> |
| <b>2 Related Work</b>                  | <b>13</b> |
| <b>3 Definitions</b>                   | <b>14</b> |
| 3.1 First-order Logic                  | 14        |
| 3.2 Quantifier Depth                   | 14        |
| 3.3 Transformer                        | 16        |
| 3.4 RASP                               | 17        |
| <b>4 Expressiveness Results</b>        | <b>18</b> |
| 4.1 Preliminaries                      | 18        |
| 4.2 Algorithms for Standard Type       | 20        |
| 4.2.1 Standard-1                       | 20        |
| 4.2.2 Standard-1 with BOS              | 21        |
| 4.2.3 Standard-2                       | 21        |
| 4.2.4 Standard-2 with BOS              | 22        |
| 4.2.5 Standard-3                       | 22        |
| 4.2.6 Standard-3 with BOS              | 24        |
| 4.3 Algorithms for Nested Type         | 24        |
| 4.3.1 Nested-1                         | 25        |
| <b>5 Data Construction</b>             | <b>26</b> |
| 5.1 Pipeline A: FOL Datasets           | 27        |
| 5.2 Pipeline B: FOL2NS Datasets        | 29        |
| 5.3 Exploration on T5                  | 30        |
| <b>6 Experimental Setup</b>            | <b>31</b> |
| 6.1 Models                             | 31        |
| 6.2 Training Details                   | 32        |

|                                               |           |
|-----------------------------------------------|-----------|
| <b>7 Results and Insights</b>                 | <b>33</b> |
| 7.1 Performance                               | 34        |
| 7.1.1 Performance on FOL                      | 34        |
| 7.1.2 Performance on FOL2NS                   | 36        |
| 7.2 Interpretation                            | 37        |
| 7.2.1 Limitations of One Layer                | 37        |
| 7.2.2 Roles of Two Layers                     | 38        |
| 7.2.3 Relations between Quantifiers           | 44        |
| 7.2.4 Key Information Aggregation             | 47        |
| 7.2.5 Other Patterns                          | 48        |
| <b>8 Discussion</b>                           | <b>49</b> |
| <b>9 Conclusion</b>                           | <b>51</b> |
| <b>Acknowledgments</b>                        | <b>51</b> |
| <b>References</b>                             | <b>51</b> |
| <b>Appendices</b>                             | <b>55</b> |
| <b>A FOL2NS Template</b>                      | <b>55</b> |
| <b>B Selection of Predicates and Entities</b> | <b>55</b> |
| B.1 Unary Predicate: 98 words                 | 55        |
| B.2 Binary Predicate: 88 words                | 56        |
| B.3 Ternary Predicate: 35 words               | 56        |
| B.4 Person: 128 words                         | 56        |
| B.5 Thing: 88 words                           | 57        |
| <b>C Plots</b>                                | <b>57</b> |

**Word count: 13589**

## List of Figures

|    |                                                                                    |    |
|----|------------------------------------------------------------------------------------|----|
| 1  | Two Pipelines for Dataset Constructions . . . . .                                  | 27 |
| 2  | Metric Comparisons on FOL . . . . .                                                | 35 |
| 3  | Confusion Matrices of a 1L2H Model . . . . .                                       | 38 |
| 4  | Attention Heatmaps of a 1L2H Model . . . . .                                       | 38 |
| 5  | Score Logits and Attention Weights Comparisons of a 2L2H Model . . . . .           | 39 |
| 6  | Value Vector Visualisation of a 2L2H Model . . . . .                               | 39 |
| 7  | Per-head Macro-average Distance of Value Vectors . . . . .                         | 40 |
| 8  | Score Logits and Attention Weights Comparisons in Layer0 of a 2L2H Model . .       | 40 |
| 9  | Attention Comparisons of a 2L2H Model on Two FOL types . . . . .                   | 41 |
| 10 | A Similar Pattern in both RASP and Transformer on <i>Standard</i> – 1 . . . . .    | 42 |
| 11 | An Example of simulating FFN in RASP . . . . .                                     | 43 |
| 12 | Two Heads in Layer3 of a 4L2H Model . . . . .                                      | 44 |
| 13 | Quantifier Cosine Similarity during Training . . . . .                             | 44 |
| 14 | Cosine of Mean Angular Distance during Inference . . . . .                         | 45 |
| 15 | An Example of Value Vector Visualisations . . . . .                                | 45 |
| 16 | A Similar Pattern in both RASP and Transformer on <i>Standard</i> – 2 . . . . .    | 46 |
| 17 | A Similar Pattern in both RASP and Transformer on <i>Standard</i> – 1 with BOS . . | 46 |
| 18 | Attention Comparisons of Two Pooling Strategies . . . . .                          | 47 |
| 19 | Attention Heatmaps of a 2L2H Model with PAD . . . . .                              | 48 |
| 20 | Attention Examples of a 2L2H Model on FOL2NS . . . . .                             | 49 |
| 21 | Same Head Roles in a 2L2H Model and a 2L4H Model . . . . .                         | 49 |
| 22 | Appendix: Full RASP Visualisation on <i>Standard</i> – 1 . . . . .                 | 57 |
| 23 | Appendix: Full RASP Visualisation on <i>Standard</i> – 2 . . . . .                 | 58 |
| 24 | Appendix: Full RASP Visualisation on <i>Standard</i> – 3 without BOS . . . . .     | 59 |
| 25 | Appendix: Full RASP Visualisation on <i>Standard</i> – 3 with BOS . . . . .        | 60 |
| 26 | Appendix: Overall Cosine of Mean Angular Distance during Inference . . . . .       | 61 |
| 27 | Appendix: Focusing on LO in L1H0 of a 4L4H Model . . . . .                         | 62 |
| 28 | Appendix: Repeated Behaviours in a 5L4H Model . . . . .                            | 63 |

## List of Tables

|   |                                                             |    |
|---|-------------------------------------------------------------|----|
| 1 | Different FOL Types . . . . .                               | 15 |
| 2 | An Example of FOL2NS Outputs . . . . .                      | 29 |
| 3 | An Example of the Lexicalisation Workflow . . . . .         | 29 |
| 4 | Symbols to Lexical Items . . . . .                          | 30 |
| 5 | Overall Model Performance on FOL . . . . .                  | 34 |
| 6 | Model Performance with smaller $d_{model}$ on FOL . . . . . | 36 |
| 7 | Overall Model Performance on FOL2NS . . . . .               | 37 |
| 8 | Appendix: Three patterns in <i>Nested</i> – 2 . . . . .     | 55 |

# Terms and Abbreviations

**NLP:** Natural Language Processing

**NLI:** Natural Language Inference

**FOL:** First-Order Logic

**QD:** Quantifier Depth

**sub\_QD:** Quantifier Depth of the subformula(s)

“ $\forall$ ”: Universal Quantifier

“ $\exists$ ”: Existential Quantifier

**LO:** Logical Operators/Connectives

**FOL2NS:** First-Order Logic to Natural Sentences

**CFG:** Context-free Grammars

**BERT:** Bidirectional Encoder Representations from Transformers

**RASP:** The Restricted Access Sequence Processing Language proposed by Weiss et al. (2021)

**S-op:** Sequence Operators in RASP

**Q:** Query

**K:** Key

**V:** Value

**QK:** Query-Key

**FFN:** Feed-forward Network(s)

**BOS / <bos>:** Beginning-of-Sequence Token

**PAD / <pad>:** Padding Token

**CLS:** Classification Token

**SEP:** Separator Token

“ $d_{model}$ ”: Embedding Dimension and Hidden Size of Transformer

**L:** Layer(s) in Transformers

**H:** Attention head(s) in Transformers

**Layer0:** The first layer in Self-attention

**Layer1:** The second layer in Self-attention

**Head0:** The first head in Self-attention

**Head1:** The second head in Self-attention

**2L2H:** Two layers and two heads

**1L1H:** One layer and one head

**1L2H:** One layer and two heads

**1L4H:** One layer and four heads

**2L1H:** Two layers and one head

**2L2H:** Two layers and two heads

**2L4H:** Two layers and four heads

**3L2H:** Three layers and two heads

**4L2H:** Four layers and two heads

**5L4H:** Five layers and four heads

**L1H2:** The third head in the second layer

**L2H0:** The first head in the third layer

**L2H2:** The third head in the third layer

**L3H0:** The first head in the fourth layer

**L4H0:** The first head in the fifth layer

**L4H1:** The second head in the fifth layer

**L4H2:** The third head in the fifth layer

# Abstract

Transformers have attracted growing interest for their abilities and limitations in modelling formal languages. However, such studies do not adequately reflect real-world texts, where models must capture the structure amid lexical elements. In this work, we treat first-order logic (FOL) formulas as a combination of  $\{a, b\}^*$  and Dyck-1 with predicates and variables, to better simulate the syntactic conditions of natural languages. Within this framework, we systematically study the abilities of Transformers to recognise quantifier depth ( $QD$ ) as well as the internal mechanism of self-attention in doing so. Specifically, we define five FOL types with increasing structural complexity and provide stepwise, interpretable RASP algorithms that simulate the full computation of a Transformer encoder on processing these formulas. We then construct seven synthetic datasets covering the five FOL types and two FOL-to-Natural-Sentence (FOL2NS) translation variants, and evaluate (i) small-sized Transformers with different configurations on the FOL datasets and (ii) BERT and a two-layer, two-head (2L2H) Transformer on the FOL2NS datasets. Empirically, two-layer Transformers perform robustly among all datasets, whereas single-layer models degrade as the structure becomes more complex. Our attention-based analyses of the 2L2H model further support an “extractor→aggregator” progression across layers and reveal additional fine-grained patterns of  $QD$  recognition. Several attention heads show behaviours that either closely mirror the visualisations of our algorithms or align with observations previously reported for larger models. We also find shortcut learning in Transformer-based architectures when processing natural sentences in FOL2NS.

## Declaration

I hereby confirm that no portion of the work referred to in the thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

## Intellectual Property Statement

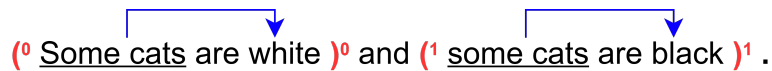
- i The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made *only* in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the thesis, for example graphs and tables (“Reproductions”), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Thesis restriction declarations deposited in the University Library, The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations/>) and in The University’s policy on Presentation of Theses.

# 1 Introduction

Based on the architecture of Transformer (Vaswani et al., 2017), language models have made remarkable progress in NLP tasks (Devlin et al., 2019; Liu et al., 2019; Raffel et al., 2023). This empirical success has attracted extensive interest in how such models handle syntax and semantics. Among all components, self-attention (Bahdanau et al., 2016; Vaswani et al., 2017) is the key innovation. A series of studies have proved that Transformers can process various kinds of formal languages via self-attention in an indirect manner, such as Dyck-k (Bhattamishra et al., 2020; Ebrahimi et al., 2020; Yao et al., 2021). In particular, context-free languages reflect key properties of human language syntax (Chomsky, 1956), providing a controlled setting to observe the models’ abilities to learn structure and the contribution of each block. However, formal languages abstract away from lexical factors and thus can not adequately model the structure of natural text, which limits their direct relevance to practical application in NLP.

To partly bridge this gap, we regard the first-order logic (FOL) formulas as a combination of two context-free languages, which offers an ideal and compact prototype with syntactic features of natural languages. Typically, predicates and variables introduce a minimal lexical environment, which is filled with uppercase and lowercase letters respectively, and can be lexicalised by real words; quantifiers, logical operators and parentheses fix the syntactic structures, which combine  $\{a, b\}^*$  ( $a = “\forall”, b = “\exists”$ )<sup>1</sup> and Dyck-1<sup>2</sup>. Moreover, as both FOL and natural language rely heavily on the notion of scope, FOL can be viewed as a simplification with fewer patterns and no ambiguity.

In this work, we define five types of first-order logic (FOL) formulas with increasing structural complexity and construct two FOL-to-Natural-Sentence (FOL2NS) mappings. We then generate seven synthetic datasets based on two automated pipelines with controlled settings. For the FOL types, we group them into two subclasses: *Standard* and *Nested*. These mirror the major structural patterns in natural-language parsing: parallel forms and nested (embedded) forms, which are related to processing long-distance and nested dependencies in English syntax. Consider “ $\exists x P(x) \wedge \exists y Q(y)$ ” as an example in the *Standard* types. Intuitively, we can connect a similar structure in English to “[*Q*] [*Noun*] [*V*] [*ADJ*] [*CONJ*] [*Q*] [*Noun*] [*V*] [*ADJ*]<sup>3</sup> and instantiate a sentence as follows:



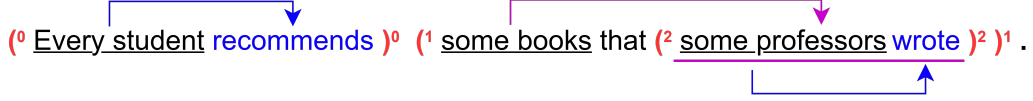
Note that this is just an analogy for structural comparison. In the actual logic-to-text

<sup>1</sup>The  $\{a, b\}^*$  language consists of all finite strings over the alphabet  $\{a, b\}$  defined by  $S \rightarrow aS \mid bS \mid \varepsilon$ .

<sup>2</sup>The Dyck-1 language over  $\Sigma = \{ (, ) \}$  consists of balanced parentheses defined by  $S \rightarrow (S) \mid SS \mid \varepsilon$ .

<sup>3</sup>“*Q*” represents the quantifier, such as “some, every, all”; “*Noun*” represents a common noun with singular or plural; “*V*” represents any verb, including auxiliaries, modals and copulas “be”; “*ADJ*” represents any gradable or non-gradable adjective; “*CONJ*” represents the conjunction, such as “and, or”.

translation, the sentence is derived from “ $\exists x(P(x) \wedge Q(x)) \wedge \exists y(P(y) \wedge R(x))$ ” and the lexicalized version “ $\exists x(Cat(x) \wedge White(x)) \wedge \exists y(Cat(y) \wedge Black(x))$ ”. Similarly, from “ $\forall xP(x) \rightarrow \exists y(Q(y) \wedge \exists zR(z))$ ” to “[Q] [Noun] [V] [Q] [Noun] [C] [Q] [Noun] [V]”<sup>4</sup>, we can get the following construction for the *Nested* types:



$$\forall x(P(x) \rightarrow \exists y(Q(y) \wedge \exists z(S(z) \wedge R(x, y, z) \wedge W(y, z))))$$

$$\forall x(Student(x) \rightarrow \exists y(Professor(y) \wedge \exists z(Book(z) \wedge Recommend(x, y, z) \wedge Write(y, z))))$$

To explore the abilities of Transformers on structural modelling, our task is framed as quantifier depth ( $QD$ ) recognition. Through nesting quantifiers in different scopes,  $QD$  reflects the logical depth of the FOL formulas and provides a good entry point to measure the sensitivity of a network to structures. This also highlights a practical challenge in NLP for quantifying the contribution of function words<sup>5</sup> in different clauses and composing them over hierarchical phrase structure. Specifically, the model needs to complete several steps: (i) analyse quantifier scope, (ii) collect the number of quantifiers in each scope, (iii) reset the counts in parallel forms or accumulate them in nested forms, and (iv) compare the aggregated information to classify the final result. During the whole process, the model should focus more on the structure, rather than being influenced by lexical interactions. As natural sentences are rarely observed to exceed three (Karlsson, 2007; Jin et al., 2018), our work controls a maximum  $QD$  at 3 to realistically model the hierarchical structure of human languages.

On the other hand, many studies have focused on the inner workings and intermediate representations of self-attention. Weiss et al. (2021) create the Restricted Access Sequence Processing Language (RASP) that formalises the computation in a Transformer encoder as abstract operations on sequences, and supplies simulated attention visualisations for the associated algorithms. It provides testable hypotheses for white-box explanations.<sup>6</sup> Moreover, the most popular approach to gain insight into self-attention internals is visualising their weights (Clark et al., 2019; Voita et al., 2019; Abnar and Zuidema, 2020). Each single attention weight quantifies the influence of one token on another.

Innovatively, our work integrates RASP algorithms, attention analysis, and value-space probing to provide a systematic interpretation on how Transformers recognise  $QD$  in FOL.

<sup>4</sup>“Q”, “Noun”, and “V” represent the same concepts as defined in the footnote 3. In addition, “C” represents the complementiser, and the word “that” is used as a relativiser.

<sup>5</sup>Our work uses quantifiers as the indicator. It can be suggestive to other words that serve a similar grammatical or structural role, such as determiners, prepositions and modals.

<sup>6</sup>White-box explanations clarify a model’s predictions by directly probing its internal structure, parameters, and computations to provide mechanism-faithful interpretations.

Our RASP algorithms show that a Transformer encoder can compute  $QD$  by using self-attention to implement the relevant operations of counting, reset, accumulation and comparison over subformulas. We then evaluate small-sized Transformers on five FOL datasets and visualise the intermediate representations of these models, and find that some patterns show strong correlations with our constructions in RASP.

For training on FOL, we consider different number of layers and attention heads, pooling strategies, and hidden sizes to analyse their individual contributions. Our results show that the models' performance is mainly influenced by the first two factors: (i) one layer struggles on complex structures, whereas two layers can manage all FOL types; (ii) mean pooling outperforms BOS-token pooling. Additionally, difficulty increases with higher  $QD$ , and the parallel form is harder than the nested one for a model to process. For FOL2NS datasets, we evaluate BERT and a 2-layer, 2-head Transformer, and find that both models hit 100% on all metrics within one epoch. This result points to potential statistical shortcuts in our FOL-to-English translation templates.

According to the attention-based analyses, we first observe a clear contrast between single-layer and two-layer transformers, which highlights a two-stage computation from extraction to aggregation. With the range of attention weights narrows after the first layer, the model effectively implements argmax-like then mean-like operations. We next probe how the model represents the relation between the universal and existential quantifiers and find that they are closely aligned with each other in the value space of Transformers. We also reveal the different mechanisms for aggregating key information under two pooling strategies. Notably, several patterns we observe, including layer-wise variation, head specialisation and redundancy, PAD tokens as auxiliary memory and shortcut learning, are also present in actual downstream tasks on larger models. It is encouraging that they are detectable even in our minimal settings.

To our knowledge, this is the first systematic study showing that small-sized Transformers can recognise quantifier depth in FOL formulas, supported by both theoretical and empirical interpretations. We contribute minimal, controlled prototypes that (i) test a network's structural sensitivity under more naturalistic conditions by treating FOL as a combination of two formal languages, and (ii) provide visualised insights for analysing Transformer-based architectures.

Overall, the paper is structured as follows: § 2 surveys related work; § 3 sets key definitions; § 4 presents our RASP algorithms in the first four FOL types; § 5 shows two pipelines for constructing synthetic datasets (FOL and FOL2NS); § 6 presents the methodology that we follow in our analyses and the models we employ; § 7 reports the empirical results of trained models, and conducts the corresponding interpretation via intermediate representations and compares with RASP visualizations; § 8 provides an overall discussion and proposes avenues for future work; § 9 concludes the dissertation.

## 2 Related Work

**Prior work on formal languages.** Formal languages, such as  $a^n b^n$  and Dyck-k, have been widely studied and recognised by recurrent architectures (Griinwald, 1996; Weiss et al., 2018; Suzgun et al., 2019). Recent works have also shown the abilities and limitations of Transformers in this domain. Bhattamishra et al. (2020) reveal that, compared to the perfect performance of LSTMs, soft-attention Transformers can process  $a^n b^n$  and a simplified variant of Dyck-k language, but have clear limitations as nesting depth increases. However, using attention heatmaps, Ebrahimi et al. (2020) find that Transformers can learn Dyck-k through interpretable stack-like patterns and surpass LSTMs. Weiss et al. (2021) further demonstrate that Transformers can fully recognise Dyck-k for any k through their Restricted Access Sequence Processing Language (RASP), which simulates self-attention and MLP layers as sequence operations. Complementary to these, we model FOL formulas as a combination of two context-free languages:  $\{a, b\}^*$  and Dyck-1, to approximate human-language syntax and evaluate Transformers’ abilities to recognise quantifier depth ( $QD$ ) under interference from predicates and variables.

**Investigating self-attention.** Recently, many studies have shown the analyses of internal representations in the self-attention block. Merrill et al. (2021) indicate that some heads attend locally to a few tokens, while others perform global averaging to facilitate counting. Clark et al. (2019) analyse BERT’s self-attention and find that certain heads are responsible for processing the syntactic relations. Voita et al. (2019) and Michel et al. (2019) also investigate the roles of different attention heads and observe that many heads can be removed during inference with little loss in performance. Abnar and Zuidema (2020) introduce attention rollout and attention flow to derive token-level relevance from attention weights. Several attention patterns observed in previous studies also emerge in our minimal two-layer, two-head Transformer. Beyond that, our approach combines the RASP algorithms with attention analyses and value-space probing to provide a systematic interpretation of the inner workings of self-attention. Moreover, Nikolaev and Padó (2023) report that sentences with the same noun participants receive high similarity scores despite having different predicates or modifiers. Motivated by this, we investigate whether nouns interfere with  $QD$  recognition by evaluating Transformer-based models on FOL2NS datasets with attention analyses.

**Connecting logic with natural languages.** Numerous studies translate natural language into logical forms. LogicNLI (Tian et al., 2021) provides a large-scale NLI dataset for multistep FOL reasoning with 20k semi-synthetic examples. FOLIO (Han et al., 2024) contains 1,430 human-annotated reasoning examples with verified FOL translations. NL2FOL (Lalwani et al., 2025) presents a neuro-symbolic pipeline that auto-formalizes arguments via the LLM-based decomposition and SMT validations. However, these datasets only view logical forms as the translation outputs and do not control quantifier

nesting in the FOL formulas. In contrast, we introduce two automated pipelines that generate synthetic datasets with progressively increasing  $QD$  through FOL templates and FOL2NS translation templates.

## 3 Definitions

### 3.1 First-order Logic

First-order logic (FOL) is a logical system for reasoning about the properties of objects or the relationships between objects in a domain. In formal semantics, it is also called “*Predicate Logic*”, an extension of propositional logic. A proposition in FOL consists of  $k$ -ary predicate symbols, quantifiers, variables, constants and function symbols. The simplest component of FOL is an *atomic formula* (e.g.,  $P(a)$ ,  $Q(a, b)$ ), which is a predicate that asserts a relationship among certain elements, i.e., the predicate refer to a property and arguments are some entities (Gamut, 1990). Importantly, *Quantification* is the key feature that separates FOL and other propositional logics. It enables FOL expressions to assert that a certain property holds for all elements with universal quantifiers (“ $\forall$ ”) or that it holds for some element with existential quantifiers (“ $\exists$ ”).

A *closed formula* is used to make the claim that some individuals satisfy a specific condition. In other words, in such formulas, each quantifier is bound with a specific variable that remains within the scope of this quantifier (Coppock and Champollion, 2025). Beyond that, non-atomic formulas are collectively referred to as *compound formulas*, using logical connectives, such as negation (“ $\neg$ ”), implication (“ $\rightarrow$ ”), conjunction (“ $\wedge$ ”) and disjunction (“ $\vee$ ”). As the negator neither changes the structure of the FOL formulas nor increases their quantifier depth, this study does not discuss it.

Our work uses the following notations: (i)  $F$  is the set of well-formed FOL formulas; (ii)  $P$  is the set of  $k$ -ary ( $k \in [1, 2, 3]$ ) predicate symbols; (iii) each  $t_m$  is a term over  $F$ , and (iv)  $a$  ranges over variable symbols. The related context-free grammar is as follows:

$$\Phi \rightarrow P(t_{1:m}) \mid (\Phi \wedge \Phi) \mid (\Phi \vee \Phi) \mid (\Phi \rightarrow \Phi) \mid \forall a \Phi \mid \exists a \Phi \quad (1)$$

### 3.2 Quantifier Depth

To measure the structural complexity of a FOL formula, one often considers its quantifier depth ( $QD$ ). As defined by Gamut (1990), the  $QD$  of a predicate-logical formula is the maximal length of a “nest” of quantifiers occurring in it. For any formula  $\varphi, \psi \in F$ , the quantifier depth  $QD(\varphi), QD(\psi)$  is defined inductively as follows:

**Rule 1.** An atomic formula has  $QD(P(t_{1:m})) = 0$ ;

| FOL Type             | Example                                                                                      | QD |
|----------------------|----------------------------------------------------------------------------------------------|----|
| Standard-1           | $\exists x \forall y \forall z P(x, y, z)$                                                   | 3  |
| Standard-2           | $\forall x P(x) \wedge \exists y \forall z R(y, z)$                                          | 2  |
| Standard-3           | $\forall x P(x) \wedge \exists y \forall z R(y, z) \rightarrow \forall w P(w)$               | 2  |
| Nested-1             | $\forall x (P(x) \vee \forall y Q(y))$                                                       | 2  |
| Nested-2 (Pattern 1) | $\forall x (P(x) \rightarrow Q(x))$                                                          | 1  |
| Nested-2 (Pattern 2) | $\exists x (P(x) \wedge \exists y (Q(y) \wedge R(x, y)))$                                    | 2  |
| Nested-2 (Pattern 3) | $\exists x (P(x) \wedge \forall (Q(y) \rightarrow \forall z (S(z) \rightarrow R(x, y, z))))$ | 3  |

**Table 1.** Examples of different types of FOL formulas discussed in this study.

**Rule 2.** For binary connectives, the depth is the maximum depth between the sides of operands:  $QD(\varphi \wedge \psi) = QD(\varphi \vee \psi) = QD(\varphi \rightarrow \psi) = \max(QD(\varphi), QD(\psi))$ ;

**Rule 3.** Each application of a quantifier increases the depth by one:

$$QD(\forall a \varphi) = QD(\exists a \varphi) = QD(\varphi) + 1.$$

To experiment under more controlled settings, we restrict our analyses to well-formed and closed FOL formulas. The following definitions enumerate all types of FOL under consideration,<sup>7</sup> and the related examples are shown in Table 1.

**Standard-1 FOL.** A non-empty quantifier prefix directly applied to a single atomic predicate, with no logical connective. Its  $QD$  is equivalent to the number of quantifiers.

**Standard-2 FOL.** It consists of two *Standard* – 1 subformulas and a single binary connective. Its  $QD$  is computed according to the *Rule 2* defined above.

**Standard-3 FOL.** It generalises *Standard* – 2 by allowing an arbitrary number of binary connectives chained at the outermost level, but still operating on *Standard* – 1 atoms. Its  $QD$  takes the maximum result of the subformula combined by connectives.

**Nested-1 FOL.** It allows arbitrary parenthesised nesting of only *Standard* – 2 expressions with binary connectives. The  $QD$  of *Nested* – 1 is based on the result of its deepest subformula and may increase with structural complexity.

**Nested-2 FOL.**<sup>8</sup> It is defined analogously to *Nested* – 1 but is restricted by the possibility of translating FOL statements to natural-sounding English sentences. It is constructed from three patterns with unary, binary and ternary predicates. Chained combinations that use no more than two binary connectives are accepted. Each pattern in *Nested* – 2 gets a fixed  $QD$  of 1 to 3.

<sup>7</sup>The evaluation of FOL expressions follows a fixed operator precedence: (i) parenthesized sub-expressions are evaluated first; (ii) logical conjunction (“ $\wedge$ ”) and disjunction (“ $\vee$ ”) take precedence over implication (“ $\rightarrow$ ”). For operators with the same precedence, evaluation proceeds from left to right.

<sup>8</sup>Nested-2 has only been tested on Transformers in Section 7, without a RASP algorithm. The templates of every pattern are listed in Table 8 in Appendix A.

### 3.3 Transformer

We consider the encoder-only model of the original Transformer (Vaswani et al., 2017), which has multiple layers with two blocks each: (i) a self-attention block and (ii) a feed-forward network (FFN). At step  $1 \leq i \leq n$ , the encoder takes an input sequence  $s_{1:i} \in \mathcal{T}^*$  ( $\mathcal{T}$  denotes the token set) and generates a sequence of vectors of the same length. Each raw token  $s_i$  is mapped to an embedding vector through the function  $f_e : \mathcal{T} \rightarrow R^{d_{model}}$ . The  $d_{model}$  is denoted as both the embedding dimension and the hidden size. For  $i \geq 1$ , let  $X_i := x_{1:i} \in R^{d_{model} \times i}$ .

**Positional Encoding.** The input vector  $x_i$  is formed by adding a positional encoding vector  $p_i$  to  $f_e(s_i)$ . Vaswani et al. (2017) propose two positional encoding schemes: (i) sine and cosine waves of different frequencies; (ii) learnable features for each position. In this work, we use the latter scheme.

**Self-Attention Block.** At layer  $l$ , the self-attention block first produces a context-aware representation at position  $i$ , which is then linearly projected to get the query, key and value vectors. Formally,

$$q_{i,l} = Q_l(x_i, l), \quad k_{i,l} = K_l(x_i, l), \quad v_{i,l} = V_l(x_i, l). \quad (2)$$

A single attention head computes attention weights over preceding positions,

$$a_{1:i,l} = softmax(\langle Q(x_{i,l}), K(x_{1,l}) \rangle, \dots, \langle Q(x_{i,l}), K(x_{i,l}) \rangle) \quad (3)$$

and the output is

$$a_{i,l} = \sum_{j=1}^i a_j^{(i,l)} v_{j,l}. \quad (4)$$

In practice, multiple heads run in parallel and their outputs are concatenated and linearly recombined to form the block's final output  $A_{i,l} \in R^{d_{model}}$ . Residual connections (He et al., 2016) and layer normalisation (Ba et al., 2016) are optionally applied around both the self-attention and feed-forward sublayers to help the network's training.

**Feed-forward Network.** The vector  $A_{i,l}$  is then passed through a position-wise feed-forward network (FFN) as  $x_{i,l+1} = O_l(A_{i,l})$ , where each  $O_l$  is a multilayer MLP with ReLU activations. Repeating this for  $l_{0:L-1}$  produces the final representation  $h_i^L$ . In addition to introducing nonlinear feature transformations, the FFN can perform element-wise operations on each component of the hidden vector at index  $i$  independently.

### 3.4 RASP

RASP (Weiss et al., 2021) provides a programming framework for specifying solutions to tasks that a Transformer encoder can learn. It also yields upper bounds on the number of layers and attention heads required to implement a given task within the model. Specifically, RASP maps different core operations, such as self-attention and feed-forward computation, into simple primitives.

**Positional Encoding.** The built-in sequence operators (S-op) *tokens* and *indices* play the same role as the combination of token embeddings and positional encodings in the Transformer encoder. As primitive inputs in RASP, they return a sequence  $S$  of length  $n$  of user-provided tokens and an auxiliary sequence of integer indices with the range from 0 to  $n - 1$ . During subsequent transformations, this information remains accessible at all times, and the process preserves the original length while performing element-wise operations. Additionally, scalars can broadcast out to the length of the current sequence. These settings simulate the explicit encoding of both content and positional information within the hidden layer.

**Self-Attention Block.** RASP realizes the self-attention block through the “*select*  $\rightarrow$  *aggregate*” mechanism. It allows information flow between sequence positions. First, the operation of  $\text{select}(q, k, p)$  mirrors the self-attention matrix. It accepts the comparison operation with a Boolean predicate  $p$  between two input sequences of length  $n$ :  $k$  as a column vector representing the key and  $q$  as a row vector for the query. It returns a selector  $Sel$ , which is a binary and unweighted selection matrix of size  $n \times n$ . For every  $i, j \in [n]$ , they satisfy  $Sel_{i,j} = p(k_i, q_j)$ . Second, the operation of  $\text{aggregate}(Sel, v)$  applies value broadcasting from a numeric sequence  $v$  to a selector  $Sel$ . It produces an output sequence  $S'$ , where each element  $s'_i$  is given by the average over the corresponding value. Rather than computing a full softmax distribution over all key-query pairs, RASP simplifies attention scores with a hard-mask pattern: it uniformly assigns the weight of  $\frac{1}{|S_i|}$  to each selected position and 0 to all others. This directly mirrors the behaviour of softmax when identical logits are given the same positive score and strongly negative logits are effectively zeroed out.

**Feed-forward Network.** To mimic the position-wise feed-forward component, RASP allows the independent computation on each element of the sequence, including mathematical operations like arithmetic, logical comparison and ternary operators. Although primitive operators contain no trainable parameters, the mathematical operations implement adaptable nonlinear mappings. This setup reflects the structure of a FFN, where a fixed activation follows linear projections.

**Other Operations.** Some built-in functions are implemented by composing more basic RASP operations. First,  $\text{selector\_width}(\text{selector})$ . It accepts a single Boolean selector and

returns a new S-op at each output index  $i$ , which produces the per-position token counts (i.e., the frequency of each symbol in the input). Second,  $\text{sort}(\text{tokens}, \text{tokens})$ , optionally including a token representing the beginning of a sequence (“\$”). In this function, each position attends to all positions with keys smaller than itself and uses  $\text{selector\_width}$  to compute its target index. Then, it uses a “ $\text{select} \rightarrow \text{aggregate}$ ” pair to move each value to its target position. Third,  $\text{indicator}(\text{bools})$ . It turns a sequence of Boolean values into numerical ones, mapping *True* to 1 and *False* to 0.

## 4 Expressiveness Results

In this section, we state theoretical results based on the RASP algorithms. It demonstrates how encoder-only Transformers can recognise  $QD$  of the *Standard 1 – 3* and *Nested – 1* FOL formulas defined in § 3.2. All *Standard* types have a variant algorithm with the prefix “\$” acting as the BOS token. Later, these algorithms also help interpret how trained Transformers recognise  $QD$ , with the related visualisations presented in Section 7 and Appendix C.

RASP provides a stepwise interpretation of how a Transformer encoder implements specific tasks. In other words, for any algorithm expressible in RASP, one can realise it in practice with an encoder-only Transformer based on the corresponding solution, potentially using a simpler configuration. Specifically, RASP can (i) maintain a counter to accumulate quantifiers via the *aggregate* operator or the *selector\_width* function, (ii) take the maximum  $QD$  via the conditional selection, and (iii) perform a reset-like operation at each logical operator when processing parallel subformulas.

In principle, the algorithms formalised in RASP can handle arbitrary  $QD$ . However, Transformer face two distinct limitations in practice. First, its working memory enforces a finite context window and bounds the architectural width. Second, a depth constraint fixes the number of layers, and the model can only perform a particular set of operations per layer. To ensure consistency with the subsequent experiments, we adopt a controlled setup for testing RASP algorithms.<sup>9</sup> All algorithms are built on the assumption that the encoder centers its computations around positional information.

### 4.1 Preliminaries

**Notation.** Let  $s_0, s_1, \dots, s_{n-1}$  denote a sequence  $S$  of input tokens of length  $n$ , where  $n \in \mathbb{Z}^+$ , and  $n - 1$  denotes the last index in the sequence. The valid positions for the keys

<sup>9</sup>The  $QD$  of test formulas in *Algorithms 1 – 4* is constrained to the range from 1 to 3; the processible depth in *Algorithm 4* is pre-defined (see details in § 4.3).

---

**Preamble 1: Basic S-ops.**

---

```
quanti = ["∀", "∃"];  
LO = ["∧", "∨", "→"];  
is_quanti = tokens in quanti;  
is_LO = tokens in LO;  
opens = (tokens == "(");  
closes = (tokens == ")");  
bos = (tokens == "§");
```

---

---

**Preamble 2: Basic Positional Selectors.**

---

```
select_all = select(1, 1, ==);  
leftward = select(indices, indices, <=);  
rightward = select(indices, indices, >=);  
equal = select(indices, indices, ==);
```

---

---

**Preamble 3: Key/Query-specific Selectors.**

---

```
k_quanti = select(is_quanti, True, ==);  
q_quanti = select(True, is_quanti, ==);  
k_LO = select(is_LO, True, ==);  
q_LO = select(True, is_LO, ==);  
k_bos = select(bos, True, ==);  
q_bos = select(True, bos, ==);  
select_quanti = k_quanti and q_quanti;  
select_LO = k_LO and q_LO;
```

---

and queries are  $i, j \in [0, n - 1]$ . The selection matrix is  $M_{i,j} \in \{0, 1\}^{n \times n}$  in each pair of positions  $(i, j)$ , where  $M_{i,j} = 1$  if the key index  $i$  is in the prefix of  $j$  (i.e.,  $i \leq j$ ), otherwise  $M_{i,j} = 0$ . The indicator function denoted as  $I(\cdot) \in \{0, 1\}$ , which returns 1 if the enclosed condition holds, and 0 otherwise. The vector  $a_j$  is the uniform average of the values on the selected keys at each query position  $j$ . We define  $D$  as the depth of the input tokens layered by the parentheses,  $QD_x$  ( $x \in [1, 6]$ ) as the quantifier depth of different FOL types, and  $sub\_QD$  as the quantifier depth in each subformula.

**Basic S-ops.** The first block lists the elementary token classes and the required sequence operators (S-ops); see Preamble 1. They provide a minimal vocabulary-level annotation that higher-level selection routines can query: (i) quantifiers (*quanti*), (ii) binary logical operators/connectives (LO), (iii) two Boolean masks over *tokens*, returning *True* at positions occupied by a quantifier (*is\_quanti*) or by a logical operator (*is\_LO*), respectively, and (iv) three further masks for identifying all structural delimiters (*opens*: opening brackets, *closes*: closing brackets, and *bos*: the beginning-of-sequence token).

**Basic Positional Selectors.** To realise different positional constraints, the second block introduces general selectors that produce a binary relation over *tokens* and *indices*; see Preamble 2. The *select\_all* is the universal relation, returning *True* at every position. Similar to the causal mask, *leftward*, *rightward* and *equal* matches each output position to all input positions satisfying the positional conditions and itself. They correspond to cases where the key index is located to the left, to the right, or exactly at the same position as the query index.

**Key/Query-specific Selectors.** With the S-ops from Preamble 1, the third preamble specialises the symmetric selectors that condition exclusively on the key side ( $k\_$ ) or the query side ( $q\_$ ); see Preamble 3. In other words, special symbols can be used as keys or as queries that aggregate information from all other tokens. The compound masks *select\_quanti* and *select\_LO* implement two-way selection to identify the locations of quantifiers or logical operators.

---

**Algorithm 1-1:** Standard-1.

---

```

def type1_fn( ){
    num_quanti = aggregate(leftward,
                          indicator(is_quanti)) ;
    QD_type1 = (indices + 1) * num_quanti ;
    return QD_type1[-1] ;
}

```

---



---

**Algorithm 1-2:** Standard-1 with <bos>.

---

```

def type1_bos_fn( ){
    QD_type1 = aggregate(k_quanti
                      and q_bos, indices) ;
    return QD_type1[0] ;
}

```

---

## 4.2 Algorithms for Standard Type

In *Standard* FOL, every formula is atomic or formed by combining subformulas with a varying number of logical operators. Here, we define each parallel subformula as a *clause*. The model can encode each clause's *QD* as a signal that increases linearly with the number of quantifiers. To delimit the quantifier scope, every logical operator needs to be located and treated as the boundary of a clause. It then aggregates the *QD* information within each segment and selects the maximum result among all clauses.

### 4.2.1 Standard-1

There exists a 2L2H Transformer that can recognize *QD* in *Standard* – 1; See Algorithm 1-1 and Figure 22a in Appendix C. Without any logical operator, this simplest type only needs to compute a sequence-level statistic from the number of quantifiers. Here,  $M$  is a leftward selection matrix. An *indicator* is applied to return 1 if  $S_i$  is a quantifier, taking the results as a value vector  $v$ .

$$M_{i,j} = I(i \leq j) \quad (5)$$

$$v_i = I(is\_quanti(s_i)) \quad (6)$$

$$v = (v_i)_{i=0}^{n-1} = (v_0, v_1, \dots, v_{n-1})^T \quad (7)$$

It then performs element-wise aggregation, averaging  $v$  on the set of keys  $S_j$  selected by its mask for each query position  $j$ . The intermediate quantity is the number of quantifiers in the selected prefix at the positions  $0 : j$  divided by its length.

$$a_j = \frac{\sum_{i=0}^{n-1} M_{i,j} v_i}{\sum_{i=0}^{n-1} M_{i,j}} = \frac{\sum_{i=0}^j v_i}{j+1} \quad (8)$$

To undo the normalisation in the previous step,  $a_j$  is multiplied by  $j+1$  (coded as *indices* + 1). The final output is extracted from the maximum value stored in the last index to get the total number of quantifiers in the entire sequence.

$$sub\_QD_j = (j+1)a_j \quad (9)$$

$$QD_1 = sub\_QD_{n-1} = na_{n-1} \quad (10)$$

**Algorithm 2-1:** Standard-2.

---

```

def type2_fn( ) {
    LO_pos = aggregate(select_LO, indices) ;
    LO_idx = aggregate(select_all, LO_pos) * length ;
    left = k_quantiti and select(indices, LO_idx, <) ;
    right = k_quantiti and select(indices, LO_idx, >) ;
    left_QD = selector_width(left) ;
    right_QD = selector_width(right) ;
    QD_type2 = left_QD if (left_QD > right_QD)
                else right_QD ;
    return QD_type2 ;
}

```

---

**Algorithm 2-2:** Standard-2 with <bos>.

---

```

def type2_bos_fn( ) {
    bos_quantiti = k_quantiti and q_bos ;
    bos_LO = k_LO and q_bos ;
    LO_idx = aggregate(bos_LO, indices) ;
    left = k_quantiti and select(indices, LO_idx, <) ;
    right = bos_quantiti and select(indices, LO_idx, >) ;
    left_QD = selector_width(left)[0] ;
    right_QD = selector_width(right)[0] ;
    QD_type2 = left_QD if (left_QD > right_QD)
                else right_QD ;
    return QD_type2 ;
}

```

---

## 4.2.2 Standard-1 with BOS

In *Standard* – 1 prefixed by “§”, there exists a 2L1H Transformer that can aggregate the processed information to the “§”; see Algorithm 1-2 and Figure 22b in Appendix C. By applying the binary selector with the conjunction of  $k_{\text{quantiti}}$  and  $q_{\text{bos}}$  (two basic selectors from Preamble 2), it simulates the process that the BOS token (“§”) attends to all positions of quantifiers and aggregates their positional indices via a uniform average. The selection matrix  $M$  has a non-zero value if  $i$  ranges over quantifier positions and  $S_j = \text{“§”}$  when  $j = 0$ . The resulting value is finally stored in BOS ( $i = 0$ ) and serves as a compact summary of the quantifier distribution.

$$M_{i,j} = k_{\text{quantiti}}(s_i) \wedge q_{\text{bos}}(s_j) \quad (11)$$

$$a_j = \frac{\sum_{i=0}^{n-1} M_{i,j} \cdot i}{\sum_{i=0}^{n-1} M_{i,j}} \quad (12)$$

$$QD_1 = a_0 \quad (13)$$

## 4.2.3 Standard-2

There exists a 3L3H Transformer that can recognize QD in *Standard* – 2; See Algorithm 2-1 and Figure 23a in Appendix C. To process *Standard*\_2, an important operation is to divide the input sequence into two clauses and compute their  $QD$  independently. To locate the single logical connective, the encoder first aggregates its information from both the key sides and the query sides to obtain its absolute position  $\delta$ . We then perform a second aggregation to distribute the index  $\delta$  across all token positions, achieved by scaling the aggregated selector by the total sequence length.

$$M_{i,j} = I(is\_LO(s_i)) \wedge I(is\_LO(s_j)) \quad (14)$$

$$\delta = a_j = \frac{\sum_{i=0}^{n-1} M_{i,j} \cdot i}{\sum_{i=0}^{n-1} M_{i,j}} \quad (15)$$

$$v_\delta = (\delta, \delta, \dots, \delta)^T \quad (16)$$

Two binary selection matrices are then constructed to isolate quantifiers on either side of this connective,

$$M_{i,j}^L = k_{\text{quanti}}(s_i) \wedge I(i < \delta), \quad M_{i,j}^R = k_{\text{quanti}}(s_i) \wedge I(i > \delta) \quad (17)$$

so that  $M^L$  (respectively  $M^R$ ) strictly selects those key positions  $i$  that are quantifiers appearing to the left (respectively right) of  $\delta$ . In this way, the procedure separately measures  $\text{sub\_}QD$  by summing over the key axis. The final output is the larger value between two  $\text{sub\_}QD$ .

$$\text{sub\_}QD^L = \sum_{i=0}^{\delta-1} M_{i,j}^L, \quad \text{sub\_}QD^R = \sum_{i=\delta+1}^{n-1} M_{i,j}^R \quad (18)$$

$$QD_2 = \max(\text{sub\_}QD^L, \text{sub\_}QD^R) \quad (19)$$

#### 4.2.4 Standard-2 with BOS

In *Standard – 2* prefixed by “§”, there exists a 3L3H Transformer that can aggregate the processed information to the “§”; see Algorithm 2-2 and Figure 23b in Appendix C. The BOS token acts as a query that simultaneously attends to every quantifier or logical connective to collect their signals. The  $\text{bos}_{LO}$  restricts attention to the only connective as seen from the BOS token, and gets the connective location  $\delta$  by aggregating over indices.

$$\text{bos}_{\text{quanti}}(i, j) = k_{\text{quanti}}(s_i) \wedge q_{\text{bos}}(s_j), \quad \text{bos}_{LO}(i, j) = k_{s_i}(i, j) \wedge q_{\text{bos}}(s_j) \quad (20)$$

$$\delta = a_j = \frac{\sum_{i=0}^{n-1} \text{bos}_{LO}(i, j) \cdot i}{\sum_{i=0}^{n-1} \text{bos}_{LO}(i, j)} \quad (21)$$

The  $M^L$  uses the same formula from Algorithm 2-1, while the  $M^R$  applies a selector based on BOS. After extracting the  $\text{sub\_}QD$  at the first position ( $s_0 = \text{“§”}$ ), the remaining steps are unchanged.

$$M_{i,j}^R = \text{bos}_{\text{quanti}}(i, j) \wedge I(i > \delta) \quad (22)$$

$$\text{sub\_}QD^L = \sum_i M_{i,0}^L, \quad \text{sub\_}QD^R = \sum_i M_{i,0}^R \quad (23)$$

#### 4.2.5 Standard-3

There exists a 5L2H Transformer that can recognize QD in *Standard – 3*; See Algorithm 3-1 and Figure 24 in Appendix C. *Standard\_3* measures the maximum  $QD$  that occurs in any top-level subformula. It shares the same underlying structure as *Standard\_2* but allows for an unbounded number of logical operators. To mark every logical operator, the

**Algorithm 3-1:** Standard-3.

---

```

def type3_fn( ){
    clause_idx = aggregate(leftward, indicator(is_LO))
                  * (indices + 1);
    same_clause = select(clause_idx, clause_idx, ==);
    freq = selector_width(same_clause);
    quanti_pos = aggregate(same_clause,
                          indicator(is_quanti));
    clause_QD = quanti_pos * freq;
    QD_type3 = sort(QD_in_clause, QD_in_clause);
    return QD_type3[-1];
}

```

---

**Algorithm 3-2:** Standard-3 with <bos>.

---

```

def type3_bos_fn( ){
    LO_bos = k_bos and q_LO;
    LO_pos = aggregate(LO_bos, 1, 0);
    quanti_bos = k_bos and q_quanti;
    quanti_pos = aggregate(quanti_bos, 1, 0);
    clause_idx = aggregate(leftward, LO_pos)
                  * (indices + 1);
    same_clause = select(clause_idx, clause_idx, ==);
    freq = selector_width(same_clause);
    clause_q_pos = aggregate(same_clause, quanti_pos);
    clause_QD = clause_q_pos * freq;
    QD = sort(QD_in_clause, QD_in_clause);
    QD_type3 = aggregate(k_bos, QD)[0];
    return QD_type3;
}

```

---

encoder first scans the input tokens from left to right using a selection matrix  $M$ , and aggregates the accumulated count of logical operators in  $a_j$ .

$$M_{i,j} = I(i \leq j) \quad (24)$$

$$a_j = \frac{\sum_{i=0}^j I(is\_LO(s_i))}{j+1} \quad (25)$$

After undoing the average value by length, it produces an ordered set  $C$  of the indices  $c_1, c_2, \dots, c_j$  that distinguishes different clauses to compute their  $sub\_QD$  separately.

$$c_j = a_j(j+1) \quad (26)$$

To get each token's relative position, we enumerate tokens sharing the same clause index and assign increasing integers by computing their frequency (i.e. the length of the current clause at each position). For example, if there are two connectives in the input sequence, then a simplified clause index can be  $[0, 0, 0, 1, 1, 1, 2, 2, 2]$ , and the relative positions for each token is  $[0, 1, 2, 0, 1, 2, 0, 1, 2]$ . To represent this procedure formally, we define the matrix  $M'$  to select the same clause, and the ordered set of relative indices as  $R$  for  $r_1, r_2, \dots, r_j$ .

$$M'_{i,j} = I(c_i = c_j) \quad (27)$$

$$r_j = \sum_{i=0}^{n-1} M'_{i,j} \quad (28)$$

Similar to Algorithm 1-1, the  $sub\_QD$  can be collected by the aggregation between  $M'$  and the value vector of a quantifier indicator. Then, undoing every result with the length provided by  $r_j$ .

$$a'_j = \frac{\sum_{i=0}^{n-1} M'_{i,j} \cdot I(is\_quanti(s_i))}{\sum_{i=0}^{n-1} M'_{i,j}} \quad (29)$$

$$sub\_QD_j = a'_j r_j \quad (30)$$

Finally, we sort all  $sub\_QD$  in ascending order so that the last element corresponds to the output of a maximum result.

$$QD_3 = \arg \max_j sub\_QD_j \quad (31)$$

#### 4.2.6 Standard-3 with BOS

In *Standard* – 3 prefixed by “§”, there exists a 6L3H Transformer that can aggregate the processed information to the “§”; see Algorithm 3-2 and Figure 25 in Appendix C. Given the BOS token as a key, it isolates the query positions of quantifiers and connectives, aggregating with a constant value of 1 to get the indicator vectors of their positions via  $a^{LO}$  and  $a^{quanti}$ .

$$LO_{bos}(i, j) = k_{bos}(s_i) \wedge q_{LO}(s_j), \quad quanti_{bos}(i, j) = k_{bos}(s_i) \wedge q_{quanti}(s_j) \quad (32)$$

$$a_j^{LO} = \frac{\sum_i LO_{bos}(i, j) \cdot 1}{\sum_i LO_{bos}(i, j)}, \quad a_j^{quanti} = \frac{\sum_i quanti_{bos}(i, j) \cdot 1}{\sum_i quanti_{bos}(i, j)} \quad (33)$$

The definitions of  $M$ ,  $M'$ ,  $c_j$  and  $r_j$  are reused from Algorithm 3-1. The difference appears in the aggregation  $a_j$  and  $a'_j$ , where we apply  $a^{LO}$  and  $a^{quanti}$  as the value vectors respectively. The subsequent operations to collect  $sub\_QD$  remain unchanged.

$$a_j = \frac{\sum_{i=0}^j a_j^{LO}}{j+1}, \quad a'_j = \frac{\sum_{i=0}^{n-1} M'_{i,j} \cdot a_j^{quanti}}{\sum_{i=0}^{n-1} M'_{i,j}} = a_j^{quanti} \quad (34)$$

Through an additional aggregation between the selector  $k_{bos}$  and the maximum  $sub\_QD$ ,

$$a''_j = \frac{\sum_{i=0}^{n-1} k_{bos}(i, j) \cdot \arg \max_j sub\_QD_j}{\sum_{i=0}^{n-1} k_{bos}(i, j)}, \quad (35)$$

the final output is encoded into the BOS token in the first position ( $s_0 = \text{“§”}$ ).

$$QD_3 = a''_0 \quad (36)$$

### 4.3 Algorithms for Nested Type

The *Nested* formulas support arbitrary parenthesized nesting of the *Standard* expressions, whose  $QD$  is incrementally accumulated across layers, beginning with the  $sub\_QD$  of the innermost subformula. The following subsections present the algorithm for *Nested* – 1.

In Preamble 4, we provide an independent algorithm for the nested depth  $D$  layered by parentheses at every position. For each position  $i$ , the cumulative counts of opening and

---

**Preamble 4:** Algorithm for layered depth.

---

```

def num_prevs(bools){
    num = aggregate(leftward, indicator(bools));
    return (indices + 1) * num;
}

n_opens = num_prevs(opens);
n_closes = num_prevs(closes);
depth = n_opens - n_closes;
adjusted_depth = depth + indicator(closes);

```

---



---

**Algorithm 4:** Nested-1.

---

```

def layer_type4(num_layer){
    layer = select(adjusted_depth, num_layer, ==);
    layer_quanti = k_quanti and layer;
    layer_LO = k_LO and layer;
    has_LO = aggregate(layer_LO, 1, 0) > 0;
    LO_idx = aggregate(layer_LO, indices);
    left = layer_quanti and select(indices, LO_idx, <);
    right = layer_quanti and select(indices, LO_idx, >);
    left_QD = selector_width(left);
    right_QD = selector_width(right);
    no_layer = aggregate(layer, 1, 0) == 0;
    only_QD = selector_width(layer_quanti);
    layer_QD = 0 if no_layer
                else
                (max(left_QD, right_QD) if has_LO
                 else (only_QD));
    return layer_QD;
}

def type4_fn( ){
    layer0_QD = layer_type4(0);
    layer1_QD = layer_type4(1);
    layer2_QD = layer_type4(2);
    QD_type4 = layer0_QD + layer1_QD + layer2_QD;
    return QD_type4;
}

```

---

closing parentheses are collected through two indicator-based aggregations, denoted as  $num[(\_)]$  and  $num[\_)]$ . The related equations can be simplified as

$$num[(\_)] = \sum_{i \leq j} I(s_i = [(\_)]), \quad num[\_)] = \sum_{i \leq j} I(s_i = [\_)]). \quad (37)$$

The layered depth  $D_i$  is computed by the difference between  $num[(\_)]$  and  $num[\_)]$ . This construction mimics a stack-like counter: each “(” pushes the stack by adding the depth by 1, and each “)” pops the stack by subtracting 1. In this way,  $D_i$  tracks the current status of the unmatched “(” symbols. We include each “)” to its corresponding depth via an additional indicator, producing the adjusted nesting depth  $D'$  for every token.

$$D_i = num[(\_)]_i - num[\_)]_i \quad (38)$$

$$D'_i = D_i + I(s_i = [\_)]) \quad (39)$$

### 4.3.1 Nested-1

There exists a 3L14H Transformer that can recognise QD in *Nested* – 1; see Algorithm 4.<sup>10</sup> In theory, *Algorithm 4* can process *Nested* – 1 at an arbitrary depth. However, RASP does not provide explicit support for recursion or “for” loops, so the maximum adjusted depth  $D'_{max}$  of the input formula must be defined in advance. After the input formula is layered by parentheses, the innermost layer contains no quantifiers (e.g., “(x)”). To reduce

---

<sup>10</sup>Since *Algorithm 4* suggests too many attention heads to visualize effectively, no corresponding figure is provided.

computational complexity, we reserve one fewer layer in each case for storing intermediate results and denote each layer as  $\ell = 0, 1, \dots, D'_{max} - 1$ . Three layers are used to cover  $D'_{max} = 4$  in testing samples of  $Nested - 1$ .

A binary matrix  $M$  is used to select all tokens  $i$  belonging to layer  $\ell$ . Within each layer, we mark the positions of the quantifiers and the only connective, yielding their layer-wise counts  $num[\ell_{quanti}]$  (denoted as  $Q$ ) and  $num[\ell_{LO}]$  by aggregation.

$$M_{i,j} = I(D'_i = \ell) \quad (40)$$

$$\ell_{quanti}(i, j) = k_{quanti}(s_i) \wedge M_{i,j}, \quad \ell_{LO}(i, j) = k_{LO}(s_i) \wedge M_{i,j} \quad (41)$$

$$Q_\ell = num[\ell_{quanti}] = \sum I(\ell_{quanti}), \quad num[\ell_{LO}] = \sum I(\ell_{LO}) \quad (42)$$

In addition, the function *layer\_type4* computes the *sub\_QD* per layer. This stage correctly processes both flat and nested structures within the following three cases:

- 1. Empty layer.** If no layer exists, then  $sub\_QD = 0$ ;
- 2. Layer with connective.** If a logical connective is present, indicating the deepest subformula, apply the same logic as processing *Standard - 2*: split the sequence at the connective, compute the number of quantifiers on each side, and take their maximum;
- 3. Nesting layer.** Otherwise, we simply count all quantifiers in that layer.

The following equation mirrors the conditional selection in the algorithm:

$$sub\_QD = \begin{cases} 0 & , \ell = \emptyset \\ \max(Q_\ell^L, Q_\ell^R) & , num[\ell_{LO}] > 0 \\ Q_\ell & , \text{otherwise.} \end{cases} \quad (43)$$

We then invoke *layer\_type4* ( $num\_layer = \ell$ ) for each layer  $\ell$  to compute and store the *sub\_QD* separately. The function *type4\_fn* sums these layer-wise results to obtain the total *QD*,

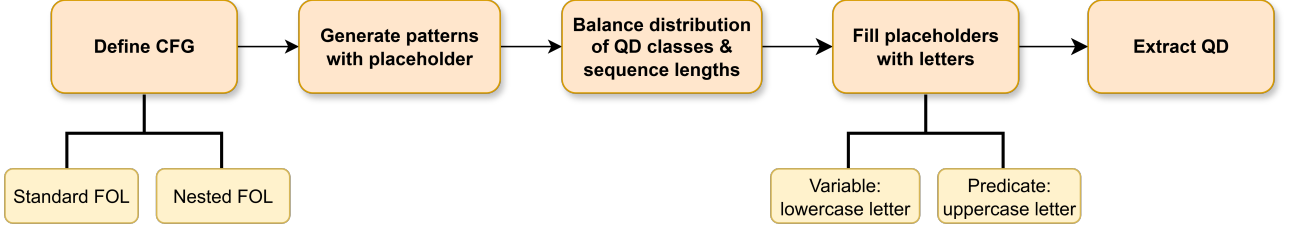
$$QD_4 = \sum_{\ell=0}^{D'_{max}-1} sub\_QD_\ell. \quad (44)$$

By increasing the range of  $\ell$ , this approach naturally extends to arbitrarily deep formulas.

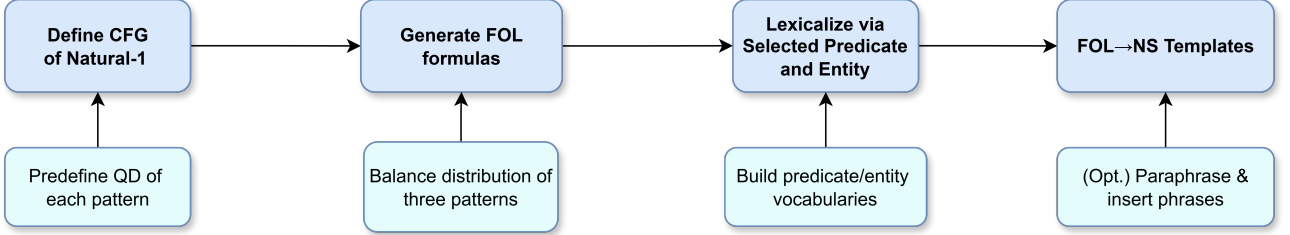
## 5 Data Construction

We propose two automated pipelines for constructing synthetic datasets, providing a controlled experimental setting with increasing *QD* complexity. Our seven datasets are generated based on these two pipelines as illustrated in Figure 1. The Pipeline A (§ 5.1)

### Pipeline A: FOL Datasets



### Pipeline B: FOL2NS Dataset



**Figure 1.** Overview of the proposed two pipelines for dataset constructions. Pipeline A generates the datasets for FOL formulas based on different types of CFG rules. Pipeline B produces both mathematical and lexicalised formats from *Nested* – 2, and then conducts the FOL2NS mapping.

generates FOL formulas covering all *Standard* types and *Nested* – 1, while the Pipeline B (§ 5.2) produces three *Nested* – 2 patterns and their variants combined by logical operators (*LO*), and maps them to the corresponding English sentences. For each type, we generate around 4,500 samples and split them into 80% for training, 10% for validation, and 10% for test.

For each formula, we bound the complexity along two factors: *QD* controls nesting depth, and the number of *LO* controls sequence length. In theory, these frameworks can increase the overall sample size by changing related parameters. They can also generate *Standard* types with arbitrary length and *Nested* – 1 with arbitrary depth. However, without an upper bound, very long sequences are challenging to manage for balanced sampling and will hinder interpretability in subsequent analyses and visualisations. Therefore, we restrict  $QD \in [1, 3]$  and  $LO \in [0, 2]$  (i.e., at most three parallel subformulas). As defined in § 3.2, we only consider closed formulas with bound variables, excluding the quantifier-free ( $QD = 0$ ) cases.

## 5.1 Pipeline A: FOL Datasets

First, we define FOL patterns with placeholders using context-free grammars (CFG). In these patterns, “#” marks predicate slots, and “\_” marks variable slots; for n-ary predicates, we also use “@” as variable placeholders. Guided by the definitions of FOL (§ 3.1) and QD (§ 3.2), we design two grammars to vary structural complexity: *Standard CFG* and *Nested CFG*. The *Standard CFG* starts from atomic formulas and repeatedly combines

logical operators from left to right, resulting in flat, two-way branches. By adjusting the maximum depth, it is generated with increasing length but without nested predicate arguments. In contrast, the *Nested CFG* allows predicate symbols to take formulas as arguments. Again, a global depth limits term-argument nesting, which jointly controls  $QD$  and overall formula complexity.

Second, we generate FOL patterns from the defined CFG rules. To obtain a balanced distribution across  $QD$  classes and  $LO$  counts, we enumerate candidate patterns at a specified depth (equal to the target  $QD$ ). We then subsample the CFG-generated pattern lists with a bounded skip strategy: select an integer  $z \in [min\_skip, max\_skip]$  at random and take every  $z$ -th formula from the ordered list (one “skip” equals one stride), which helps control the pattern style. In addition, we constrain the length of each sequence with a pre-defined range of  $[min\_len, max\_len]$ .

Third, we instantiate concrete formulas by replacing placeholders in the syntax of well-formed FOL:

- 1. Quantifier binding.** Every quantifier introduces a bound variable that is only accessible within its scope;
- 2. No duplication.** Duplicate subformulas are not allowed;
- 3. Filling placeholders.** We select predicate symbols from the 26 uppercase letters (A-Z) in the English alphabet and select variables from the 26 lowercase letters (a-z). As our task is to probe Transformers’ sensitivity to structural patterns, which is independent of the lexical choice, we instantiate placeholders with as many different letters as possible. This encourages the model to understand the structure and reduces bias from some specific tokens;
- 4. Arity separation.** In any formula, predicates of the same arity (unary, binary, ternary) may repeat or vary, but predicate symbols are not shared across arities (i.e., the uppercase letters for different arities are disjoint).

Finally, we apply a stack-based parser to extract  $QD$  and scope. Given a formula string, the parser returns the maximum  $QD$  and the span-level annotations for each quantifier. It performs a single left-to-right pass, detects quantifier-variable pairs (“ $\forall$ ” or “ $\exists$ ” followed by a letter), and groups consecutive pairs into a quantifier chain. For each chain, the current stack height (push on “(”, pop on “)”) defines the nesting depth, and the scope is delimited by the matching parentheses. The maximum stack height over the pass is taken as the global  $QD$ .

| Type                  | Output                                                                         |
|-----------------------|--------------------------------------------------------------------------------|
| CFG-based FOL pattern | $\exists\_(\#(\_) \wedge \forall\_(\#(\_) \rightarrow \#(\_, \_)))$            |
| FOL Formula           | $\exists x(P(x) \wedge \forall y(Q(y) \wedge R(x, y)))$                        |
| Lexicalized FOL       | $\exists x(Professors(x) \wedge \forall y(Students(y) \wedge Motivate(x, y)))$ |
| Natural Sentence      | <i>Some professors motivate all students.</i>                                  |
| Paraphrased Sentence  | <i>It is some professors who motivate all students with good care.</i>         |

**Table 2.** An example of output FOL2NS of *Nested* – 2 (Pattern 2) in every step. The process begins with a CFG-based FOL pattern containing placeholders, which are subsequently instantiated with variables and predicates to produce both mathematical and lexicalized forms. The corresponding natural sentence and its paraphrased variant are generated using distinct templates.

| Symbol             | Lexical Item                                                                                                             |
|--------------------|--------------------------------------------------------------------------------------------------------------------------|
| Original Formula   | $\forall x(P(x) \rightarrow \forall(Q(y) \rightarrow \forall z(S(z) \rightarrow R(x, y, z))))$                           |
|                    | P $\rightarrow$ “ <i>Person</i> <sub>1</sub> ”: Professors                                                               |
| Sampling           | Q $\rightarrow$ “ <i>Person</i> <sub>2</sub> ”: Students                                                                 |
| Entity & Predicate | S $\rightarrow$ “ <i>Thing</i> ”: Books                                                                                  |
|                    | R $\rightarrow$ “ <i>Ternary Predicate</i> ”: Explain                                                                    |
| Final Output       | $\forall x(Professors(x) \rightarrow \forall(Students(y) \rightarrow \forall z(Books(z) \rightarrow Explain(x, y, z))))$ |

**Table 3.** An example of the lexicalisation workflow of *Nested* – 2 (Pattern 3).

## 5.2 Pipeline B: FOL2NS Datasets

The second pipeline generates paired logical and natural-language data from three *Nested* – 2 patterns. An example of the outputs in FOL2NS (Pattern 2) is shown in Table 2. Similar to the Pipeline A, we start with placeholder patterns and regulate the distribution of LO-combined variants across sequence lengths. By construction, each pattern’s *QD* aligns with its predicate arity (e.g., Pattern 3 with a ternary predicate has *QD* = 3). Placeholders are then replaced as above to produce fully specified FOL formulas.

Moreover, we incorporate real-world lexical items into the positions of predicates and entities, and fill variable slots with an ordered set of lowercase letters<sup>11</sup>, thereby converting the formulas into concrete, readable formats. We define five custom vocabularies (see details in Appendix B), including two entity sets (“*Person*” with 128 occupations and “*Thing*” with 88 object names), and three predicate sets (98 unary predicates, 88 binary predicates and 35 ternary predicates). For each pattern, we uniformly sample from the required vocabularies according to the predicate arity and entity class. A step-by-step example of the process is shown in Table 3. The next step is to map lexicalised formulas to English sentences. As disambiguation is central in logic-to-text tasks, we use three fixed templates to enforce the deterministic and one-to-one mapping. Special symbols are translated as specific lexical items according to Table 4. The base templates are:

**Pattern 1.** “[*Q*] [*Person*] are [*UP*].”;

<sup>11</sup>To  $\{x, y, z, u, v, w, p, q, r\}$ .

| Symbol        | Lexical Item         |
|---------------|----------------------|
| $\forall$     | all                  |
| $\exists$     | some                 |
| $\wedge$      | and                  |
| $\vee$        | or                   |
| $\rightarrow$ | , which implies that |

**Table 4.** Mapping relationships between symbols and lexical items.

**Pattern 2.** “[ $Q$ ] [ $Person$ ] [ $BP$ ][ $Q$ ] [ $Person$ ].”;

**Pattern 3.** “[ $Q$ ] [ $Person$ ] [ $TP$ ][ $Q$ ] [ $Thing$ ] to [ $Q$ ] [ $Person$ ].”,

where “ $Q$ ” ranges over  $\{all, some\}$  and “ $UP$ ” / “ $BP$ ” / “ $TP$ ” denote unary/binary/ternary predicates. For the variants combined with logical operators, we obtain the related natural sentences by joining the base forms with connectives from  $\{and, or, implies\}$ . Take an example of Pattern 1, “ $\forall x(Professors(x) \rightarrow Knowledgeable(x)) \wedge \exists y(Students(y) \wedge Diligent(y))$ ” maps to “*All professors are knowledgeable and some students are diligent.*”

To increase surface-form diversity and reduce models’ reliance on fixed templates, we further apply controlled paraphrasing via cleft constructions and adverbial insertions in the middle or at the end of the sentence. The focus constructions include the emphatic sentence (“*it is [Q] [Person] who ...*”) and the dummy subject with a clause (“*it is the case that...*”). The insertions contain different temporal and frequency adjuncts.<sup>12</sup> This step preserves the original semantics as well as their argument structures,  $QD$ , quantifier scopes and polarities.

### 5.3 Exploration on T5

We also fine-tune T5 on a preprocessed dataset from the existing FOLIO (Han et al., 2024) corpus to translate FOL formulas into natural sentences. However, due to the limited number of samples with increasingly nested  $QD$  in the original corpus, the T5 model tends to compress information when generating sentences with longer sequence lengths, sometimes altering the structure by reducing the number of quantifiers. Although this attempt does not yield the desired results at the time, the findings suggest that T5 has a strong potential for logic-to-text tasks, with the ability to ensure both formal correctness and natural fluency.

<sup>12</sup>e.g., after careful planning, with great care, effectively, regularly, without exception, occasionally, in a timely manner, by the end of the day.

## 6 Experimental Setup

Our constructions in Section 4 show that the self-attention block can help the model recognise  $QD$  in four types of FOL formulas. We now bridge these theoretical insights with experiments to explore whether an actual Transformer-based architecture can learn to predict  $QD$  from finite samples and to interpret its working mechanism.

First, we train small-sized Transformers on the FOL datasets defined in Section 5 and systematically investigate the effect of different pooling strategies, number of layers (L) and heads (H), and hidden size on the model performances, as well as visualise the attention weights to track their internal behaviours. For the 2-layer, 2-head (2L2H) Transformer with a hidden dimension ( $d_{model}$ ) of 8, we further extract attention logits and value vectors. Second, to evaluate symbolic reasoning across different model scales, we fine-tune a vanilla pre-trained BERT on the FOL2NS datasets based on *Nested* – 2 and benchmark its performance against that of a 2L2H Transformer trained on the same data. We have made our source code available at [https://github.com/Mifuxuanan/LELA71000\\_Transformers-Recognise-QD](https://github.com/Mifuxuanan/LELA71000_Transformers-Recognise-QD).

### 6.1 Models

Our main experiments focus on the models of small scale to avoid their memorisation of the training data and make it feasible to visualise the attention patterns. As defined in § 3.3, we use an encoder-only Transformer of the original seq-to-seq architecture (Vaswani et al., 2017). To convert contextual embeddings into  $QD$  labels, we add a single classification head with softmax. The final hidden vector of the encoder at position  $i$  is represented as  $h_i^L \in R^{d_{model}}$ . Let the number of target  $QD$  classes be  $C_q$  (e.g.  $C_q = 3$  for the set  $\{1, 2, 3\}$ ), and  $c_q$  or  $c'_q$  ranges over the individual class indices  $1, 2, \dots, C_q$ . For the classifier, we introduce a learnable linear map to produce the class logits, apply softmax to obtain the probabilities of all classes, and then get the output  $\hat{y}_i$ .

$$W \in R^{C_q \times d_{model}}, b \in R^{C_q} \quad (45)$$

$$z_i = Wh_i^L + b \quad (46)$$

$$P_i = P(y_i = c_q) = \frac{\exp(z_i, c_q)}{\sum_{c'_q=1}^{C_q} \exp(z_i, c'_q)} \quad (47)$$

$$\hat{y}_i = \arg \max_{c_q} P_i \quad (48)$$

In summary, for an input sequence  $s_1, s_2, \dots, s_i$ , the model needs to (i) process the sequence at each step  $i$ , (ii) assign a probability to each class, and (iii) output the most likely class label.

In our initial few experiments, we built an encoder from scratch with Kaiming uniform (He et al., 2015) at a wider range ( $\approx \pm\sqrt{6}/\sqrt{fan\_in}$ ) than PyTorch’s Linear default ( $\approx \pm 1/\sqrt{fan\_in}$ ).<sup>13</sup> Although it converges more rapidly and matches the performance of the standard implementation, its attention-weight visualisations are too disorganised to support meaningful analysis. Given that our research objective focuses on interpretability, we decided to go ahead with the encoder layer provided by PyTorch, which yields clearer and more consistent attention patterns in the majority of experiments.

We use a regex-based tokeniser and construct a custom vocabulary of 62 tokens, including all necessary special symbols in our FOL formulas and different placeholders for predicates and variables.<sup>14</sup> Each token is then mapped to an ordered integer index to be used as input ids. We also remove all commas in every sample, preventing the models from finding a statistical shortcut from this symbol that is irrelevant to  $QD$  recognition. In some experiments, we prepend the special symbol “\$” as a BOS marker. Similarly, we map another vocabulary consisting of a fixed set of 480 English words<sup>15</sup> required for training the Transformer on the FOL2NS datasets.

Additionally, we fine-tune a “base” sized BERT model with  $d_{model} = 758$ , 12 layers and 12 attention heads on the samples of FOL2NS. BERT is pre-trained on 3.3 billion tokens of unlabeled text using Masked Language Modelling and Next Sentence Prediction (Devlin et al., 2019), and is commonly used for fine-tuning or feature extraction on downstream tasks. When using Hugging Face’s AutoTokenizer, each input sequence is automatically prefixed with a special token [CLS], and another token [SEP] is added at the end. We then feed the [CLS] representation into a linear classification head with softmax to predict the target labels. As our task is sequence-level classification (one label per input), we do not need to merge the subword tokens generated by the tokeniser into word-level representations before classification.

## 6.2 Training Details

In our main experiments, we consider Transformers with (i) mean pooling and BOS-token pooling, (ii) layer depth  $L \in \{1, 2, 3\}$ , (iii) number of attention heads  $H \in \{1, 2, 4\}$ , and (iv) hidden dimension  $d_{model} \in \{8, 16, 256\}$ . For training the small models, we use the AdamW optimiser, a batch size of 32, a learning rate of 1e-5 and no dropout. For the configuration with  $d_{model} = 256$ , we train for up to 30 epochs and employ early stopping with a patience of 10 epochs. All other configurations are trained for up to 100 epochs with early stopping

<sup>13</sup>The “*fan\_in*” is used to set the weight-initialisation scale, representing the number of inputs feeding each output unit in a layer.

<sup>14</sup>The FOL vocabulary includes “<pad>”, “\$”, “ $\forall$ ”, “ $\exists$ ”, “ $\wedge$ ”, “ $\vee$ ”, “ $\rightarrow$ ”, “(”, “)”, and all lowercase and uppercase letters.

<sup>15</sup>All English words in the FOL2NS vocabulary are defined in Section 4 and shown in Appendix B.

after 15 epochs of no improvement. The accuracy of validation set is used as the metric for checkpoint selection. In some experiments, we also try configurations with more layers and heads. The related results are shown in Section 7.

Although the nature of QD is more suitable for a regression task, we design it as a supervised multiclass classification task to better control and simplify both experiments and evaluations. We train the models on the synthetic datasets of different FOL types described in Section 5, where each sample pairs a generated formula with its corresponding  $QD$  label. The learning objective of the model is to minimise the cross-entropy between the predicted probability and the true label. We report overall accuracy, macro-averaged F1 and F1 scores in each  $QD$  class (per-class F1) as evaluation metrics at both the validation and test stages. The accuracy of the model over the samples is the fraction of total samples predicted correctly. Macro F1 computes the unweighted mean of the per-class F1 scores, assigning equal importance to each class. We also report the per-class F1 to provide a detailed view of how the model performs at individual  $QD$  levels, as there is a positive correlation between the  $QD$  labels and the formula complexity. We consider a model to perfectly recognise  $QD$  of a given FOL dataset if it achieves 100% on the above three metrics.

We extract attention weights, score logits and value vectors from Transformers under the above configurations on both validation and test sets, and visualise them as heatmaps. In preliminary trials, we retained all PAD tokens at the query side, yielding several noteworthy patterns. To produce clearer and more interpretable plots, we truncate PAD in both axes of the heatmap in most experiments. Additionally, we try to quantify the relation between “ $\forall$ ” and “ $\exists$ ” on the training and validation datasets. Accordingly, we (i) track the cosine similarities between their lexical embeddings across training epochs, and (ii) compute the cosine of pairwise-mean angular distance<sup>16</sup> based on their value vectors on the first 32 validation samples, excluding cases with only a single quantifier type.

For fine-tuning on FOL2NS, we adopt the following hyperparameters for BERT: 32 batch size,  $1e-5$  learning rate,  $d_{model} = 768$  and 0.2 dropout rate. Training is conducted for up to 10 epochs under the AdamW optimiser, with early stopping at 3 epochs.

## 7 Results and Insights

This section is organised into two main parts. The first part reports the experimental results alongside the corresponding analyses (§ 7.1), focusing on the performance of Transformer-based architectures to recognise different logical structures in mathematical

---

<sup>16</sup>We take all “ $\forall$ - $\exists$ ” token pairs within a sample, compute and average their angular distance, and convert it back to the cosine value.

| Layer(L) & Head(H) | Mean Pooling |          |               | BOS-token pooling |          |               |
|--------------------|--------------|----------|---------------|-------------------|----------|---------------|
|                    | Accuracy     | Macro F1 | Per-class F1  | Accuracy          | Macro F1 | Per-class F1  |
| <b>Standard-1</b>  |              |          |               |                   |          |               |
| 1L1H               | 100          | 100      | 100, 100, 100 | 74                | 72       | 78, 70, 68    |
| 1L2H               | 100          | 100      | 100, 100, 100 | 100               | 100      | 100, 100, 100 |
| 2L1H               | 100          | 100      | 100, 100, 100 | 100               | 100      | 100, 100, 100 |
| 2L2H               | 100          | 100      | 100, 100, 100 | 100               | 100      | 100, 100, 100 |
| <b>Standard-2</b>  |              |          |               |                   |          |               |
| 1L2H               | 81           | 80       | 99, 68, 73    | 83                | 83       | 87, 80, 81    |
| 2L1H               | 99           | 99       | 99, 100, 100  | 100               | 100      | 100, 100, 100 |
| 2L2H               | 100          | 100      | 100, 100, 100 | 100               | 100      | 100, 100, 100 |
| 3L2H               | 100          | 100      | 100, 99, 100  | 100               | 100      | 100, 100, 100 |
| <b>Standard-3</b>  |              |          |               |                   |          |               |
| 1L2H               | 48           | 48       | 51, 46, 48    | 70                | 67       | 86, 61, 54    |
| 2L1H               | 99           | 99       | 99, 100, 100  | 100               | 100      | 100, 100, 100 |
| 2L2H               | 100          | 100      | 100, 100, 100 | 100               | 100      | 100, 100, 100 |
| 3L2H               | 100          | 100      | 100, 100, 100 | 100               | 100      | 100, 100, 100 |
| <b>Nested-1</b>    |              |          |               |                   |          |               |
| 1L1H               | 100          | 100      | 100, 100, 100 | 82                | 81       | 86, 79, 78    |
| 1L2H               | 100          | 100      | 100, 100, 100 | 84                | 83       | 91, 74, 83    |
| 2L1H               | 100          | 100      | 100, 100, 100 | 100               | 100      | 100, 100, 100 |
| 2L2H               | 100          | 100      | 100, 100, 100 | 100               | 100      | 100, 100, 100 |
| <b>Nested-2</b>    |              |          |               |                   |          |               |
| 1L2H               | 84           | 84       | 97, 78, 76    | 48                | 46       | 48, 40, 50    |
| 1L4H               | 88           | 87       | 93, 82, 85    | 85                | 84       | 91, 77, 85    |
| 2L1H               | 99           | 99       | 100, 99, 99   | 100               | 100      | 100, 100, 100 |
| 2L2H               | 99           | 99       | 100, 99, 99   | 100               | 100      | 100, 100, 100 |

**Table 5.** Under 30 epochs, performance of Transformers with  $d_{model} = 256$  across different types of FOL formulas. Per-class:  $QD = 1$ ,  $QD = 2$ ,  $QD = 3$ .

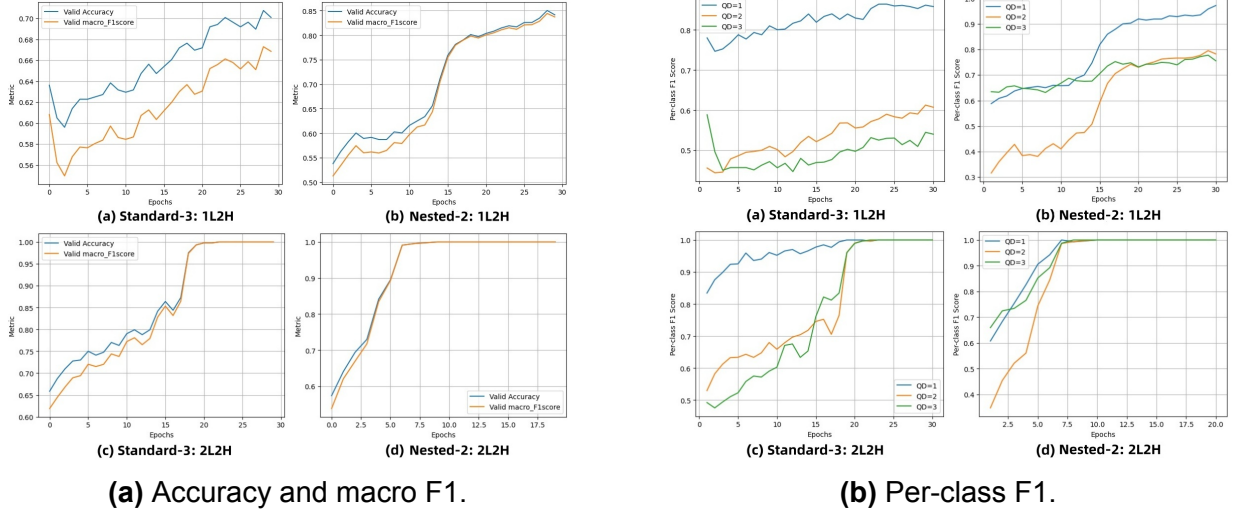
formulas and natural sentences. In the second part, we conduct interpretability exploration (§ 7.2), explaining important phenomena through attention patterns to reveal the underlying factors driving these outcomes.

## 7.1 Performance

We evaluate small-sized Transformers with different configurations on five FOL types (§ 7.1.1) as well as BERT and a 2L2H Transformer on the two FOL2NS types (§ 7.1.2), using accuracy, macro-F1 and per-class F1 scores as metrics.

### 7.1.1 Performance on FOL

With different numbers of layers (L) and heads (H), Table 5 shows the abilities and limitations of each model under two pooling strategies, when fixing the hidden dimension at 256. Table 6 reports the results of the 2L2H model with reduced hidden sizes of 8 and 16.



**Figure 2.** The validation metrics of a 1L2H Transformer (top) and a 2L2H Transformer (bottom) across epochs on *Standard* – 3 and *Nested* – 2.

First, single-layer Transformers deteriorate in complex structures, while two layers enable the model to handle all FOL types. This effect is particularly obvious in *Standard* – 3 and *Nested* – 2 under mean pooling (Figure 2a), with accuracy jumping from 48% and 84% to 99% as the model converts from a single-layer to a two-layer configuration. Moreover, increasing the number of attention heads contributes less than additional layers. In *Nested* – 2, the 1L4H model with mean pooling gains only 3-4 points on each metric than the 1L2H model, while the 2L1H model improves 15 points. A more pronounced difference between these two configurations (1L4H: +37-38%; 2L1H: +52-54%) emerges when applying BOS-token pooling.

Second, Transformers face greater challenges as the structural complexity or quantifier depth increases. Consistent with our design, the models require more computations to understand the progressive difficulties of various tasks. Under the 1L2H setting, the model achieves perfect accuracy and macro-F1 on the simplest tasks (*Standard* – 1 and *Nested* – 1), but the two metrics drop to 81% and 80% on *Standard* – 2, to 84% for both on *Nested* – 2, and to 48% for both on *Standard* – 3. The per-class analyses track the same trend: higher-QD classes are mainly responsible for the drop in overall performance. For example, the 1L2H model with mean pooling gets per-class F1 at 99%, 68% and 73% in *Standard* – 2. With further details in Figure 2b, we reveal that (i) low-QD samples ( $QD = 1$ ) are learned rapidly, enabling the models reach high scores in early training; (ii) even with multilayers, the models struggle increasingly as the quantifiers in each subformula grow; (iii) samples with  $QD = 2$  belong to the hardest class under most conditions.

Third, the pooling strategy<sup>17</sup> plays a critical role in the model performance, with mean

<sup>17</sup>For mean pooling, we get the average value of every token in the final layer as the representation for the whole sequence; for BOS-token pooling, we only provide the embedding of the BOS token to the classifier head.

| FOL Type   | $d_{model} = 8$ |          |               | $d_{model} = 16$ |          |               |
|------------|-----------------|----------|---------------|------------------|----------|---------------|
|            | Accuracy        | Macro F1 | Per-class F1  | Accuracy         | Macro F1 | Per-class F1  |
| Standard-1 | 100             | 100      | 100, 100, 100 | 100              | 100      | 100, 100, 100 |
| Standard-2 | 93              | 92       | 96, 88, 93    | 100              | 100      | 100, 100, 100 |
| Standard-3 | 86              | 85       | 94, 84, 77    | 100              | 100      | 100, 100, 100 |
| Nested-1   | 97              | 97       | 99, 96, 97    | 100              | 100      | 100, 100, 100 |
| Nested-2   | 98              | 98       | 100, 98, 98   | 100              | 100      | 100, 100, 100 |

**Table 6.** Under 100 epochs, performance of 2L2H Transformers with mean pooling strategy across different FOL types. Per-class:  $QD = 1$ ,  $QD = 2$ ,  $QD = 3$ .

pooling generally working better. A single-layer Transformer with BOS-token pooling performs poorly on all tasks, while mean pooling enables that model to successfully recognise *Standard - 1* and *Nested - 1*. In most cases, changing to BOS-token pooling reduces all metrics, with the sole exception of *Standard - 3*, where it improves the accuracy of a 1L2H model from 48% to 70%. For deeper architectures, BOS-token pooling narrows the gap, but mean pooling still yields uniformly perfect scores.

Fourth, all mean-pooling models perform worst on *Standard - 3*, suggesting that the parallel form is more challenging to recognise than nesting. This phenomenon follows the nature of different FOL types. With more parallel subformulas, the model must compute and maintain counts for each branch separately before combining them, increasing interference and the memory load. By contrast, the scope decision in the nested structure reduces to a local comparison at the innermost depth. The quantifier count can be accumulated monotonically with the current nesting level. Additionally, more parentheses in that structure provide clearer cues to determine boundaries and make the related task easier.

Finally, the hidden dimension does not appear to be the bottleneck. Although smaller  $d_{model}$  values require more epochs for the model to fully converge, their representational capacity remains sufficient. Combining the results in Tables 5 and 6, we find that configurations with  $d_{model} \geq 16$  consistently achieve 100% accuracy and macro-F1 scores among all cases.

Even with  $d_{model} = 8$ , a 2L2H Transformer with mean pooling gets near-perfect performance ( $\geq 97\%$ ) on three simpler types. Similar to the above observation, the limited hidden width primarily affects tasks with higher  $QD$  or structural complexity, while simpler tasks remain largely unaffected.

### 7.1.2 Performance on FOL2NS

On the FOL-based natural sentences, both our 2L2H Transformer and fine-tuned BERT achieve 100% across all metrics within a single epoch (Table 7). The same holds for the paraphrased dataset, which permutes the clause structure and inserts various modifiers

| FOL2NS Type          | BERT     |          |               | 2L2H Transformer |          |               |
|----------------------|----------|----------|---------------|------------------|----------|---------------|
|                      | Accuracy | Macro F1 | Per-class F1  | Accuracy         | Macro F1 | Per-class F1  |
| Natural Sentence     | 100      | 100      | 100, 100, 100 | 100              | 100      | 100, 100, 100 |
| Paraphrased Sentence | 100      | 100      | 100, 100, 100 | 100              | 100      | 100, 100, 100 |

**Table 7.** Performance of BERT and 2L2H Transformers on two FOL2NS types. Per-class:  $QD = 1$ ,  $QD = 2$ ,  $QD = 3$ .

that change quantifier positions in different sentences. This suggests that Transformer-based architectures can learn to ignore peripheral parts and find stable shortcuts from our generated templates, rather than performing deep analyses.

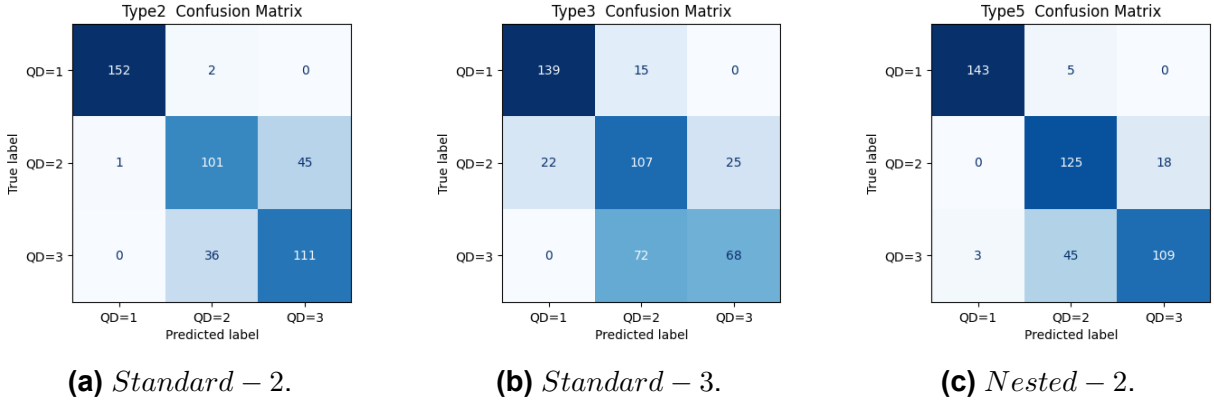
## 7.2 Interpretation

In this section, we attempt to provide an explanation of the reported results using attention patterns. We have some preliminary findings after analysing the self-attention internals with attention weights, QK score logits (pre-softmax logits) and value vectors. The default model discussed here is a 2L2H Transformer with mean pooling, as it represents the most compact and general model for comparing behaviours among different layers and heads. Other configurations are considered when interpreting specific phenomena. We primarily analyse the models with  $d_{model} = 256$ , which yield clearer and more interpretable patterns. Only for observing value vectors, we fix  $d_{model} = 8$  to facilitate visualisations, and each head is assigned four dimensions in a 2L2H model.

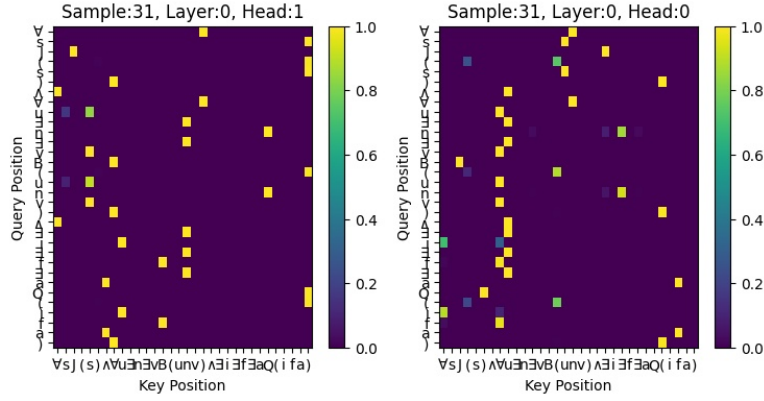
### 7.2.1 Limitations of One Layer

The results in Table 5 show that a single-layer network underperforms as the structural complexity of FOL formulas increases. After extracting the classifier logits, we find that the top-1 logit ( $\approx 0.48$ ) and top-2 logit ( $\approx 0.45$ ) present a small margin of only 0.03 across all error cases. According to the confusion matrices of three complex FOL types (Figure 3), the dominant error mode is a boundary confusion between adjacent classes, with  $QD = 3$  more often underestimated as  $QD = 2$ . These findings indicate that a single-layer Transformer struggles with choosing from multiple counting results in each subformula.

Except for *Standard* – 1, we require the Transformer to implement a reset-like operation when encountering every logical operator. Similarly, Bhattamishra et al. (2020) report the limitation of a single-layer Transformer on processing Reset-Dyck-1 Language, and explain that with only one layer, both the attention score  $\langle Q(x_n), K(\cdot) \rangle$  and the value vector at the reset symbol are independent of the preceding inputs, which cannot provide reset computations. Beyond that, we observe a noisy attention pattern related to the underlying



**Figure 3.** Confusion Matrices of a 1L2H Model on three FOL types.

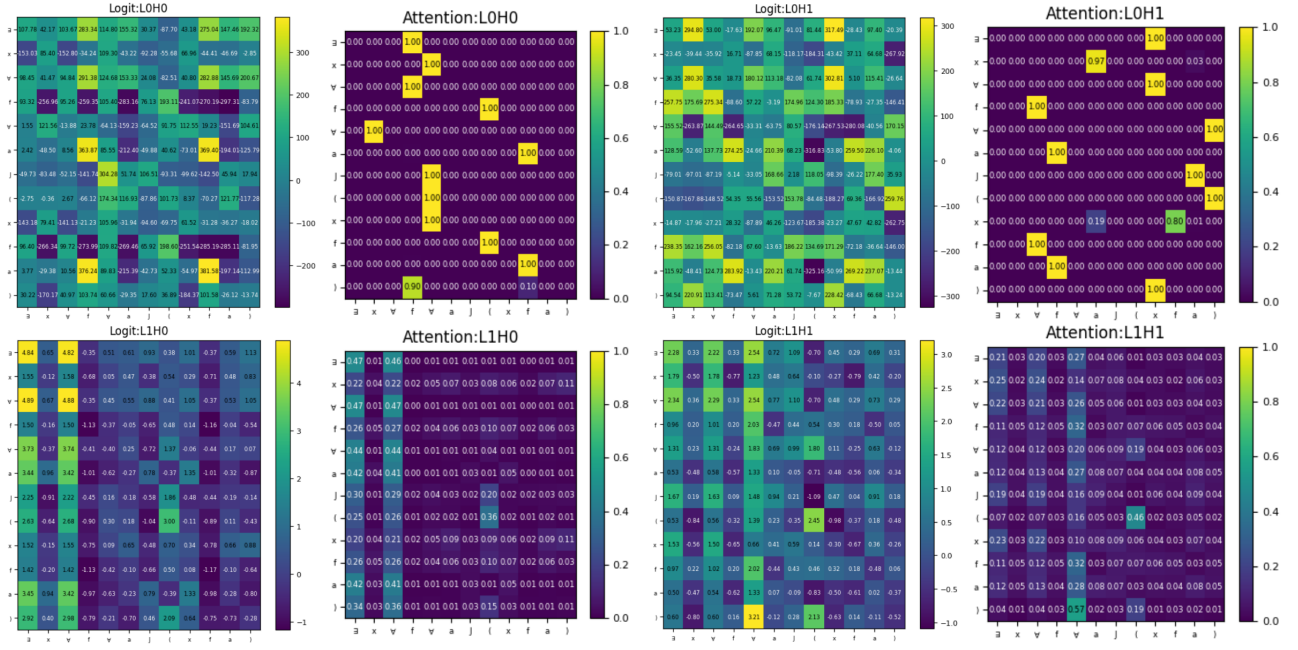


**Figure 4.** A heatmap example of a 1L2H Transformer on *Standard - 3* with  $QD = 3$ .

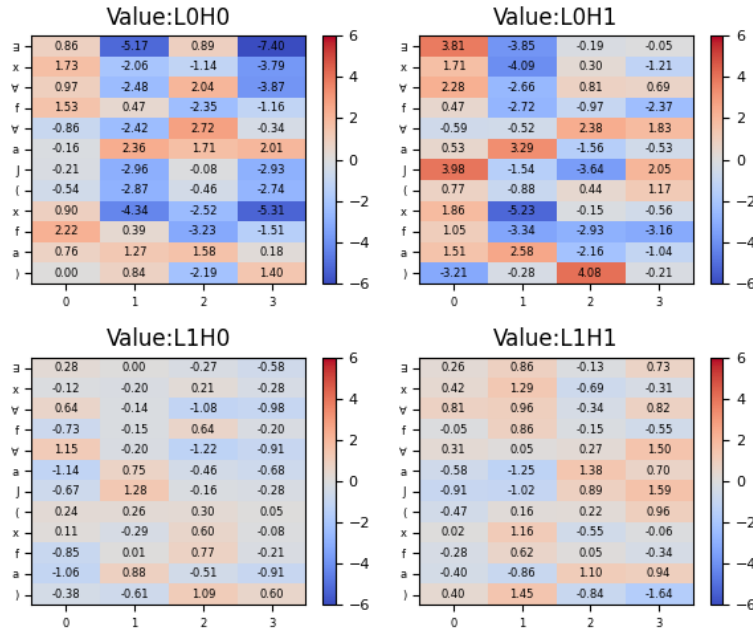
reason. See an example on *Standard - 3* (Figure 4). At the only layer, the model aims to find relations among tokens, including parenthesis matching, quantifier–quantifier pairing, quantifier–variable binding and so on, but it often extracts wrong relationships or partial cues. Therefore, the model needs another layer to double-check whether the selected pair is useful or not and which part should aggregate the information. Moreover, a single layer lacks the multiple rounds of non-linear adjustment offered by the feed-forward networks (FFN) and other components.

### 7.2.2 Roles of Two Layers

As described in § 7.1.1, two-layer Transformers can easily recognise all FOL types. Consistent with this observation, a layer-wise pattern emerges in that kind of models. At Layer0, the attention-related signals of each token span a wide range, while at Layer1, they are smoothed with more uniform values. Specifically, Layer0 preserves more original input features and assigns values with a large gap to different tokens. This enables the model to distinguish tokens from each other in a wide dynamic range and to better capture the local patterns. In contrast, Layer1 smooths these signals by reducing extremes and distributing more uniform attention everywhere. In this step, the model tends to treat tokens as equivalent. The two layers are close to implementing the argmax and mean



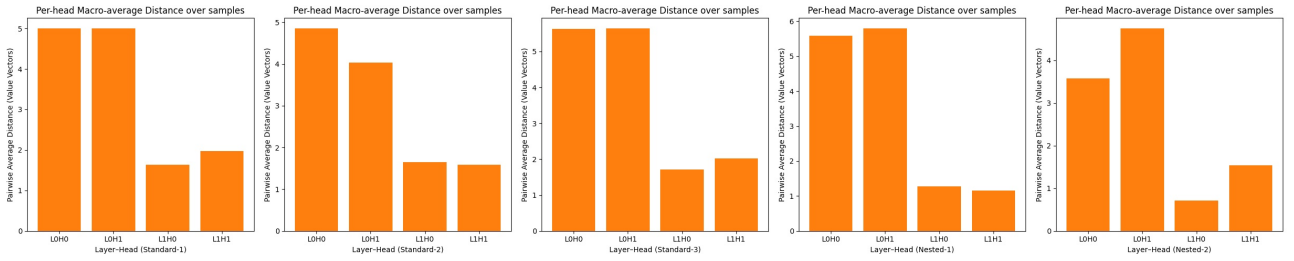
**Figure 5.** A heatmap comparison between score logits and attention weights of a 2L2H Transformer on *Standard* – 1 with  $QD = 3$ .



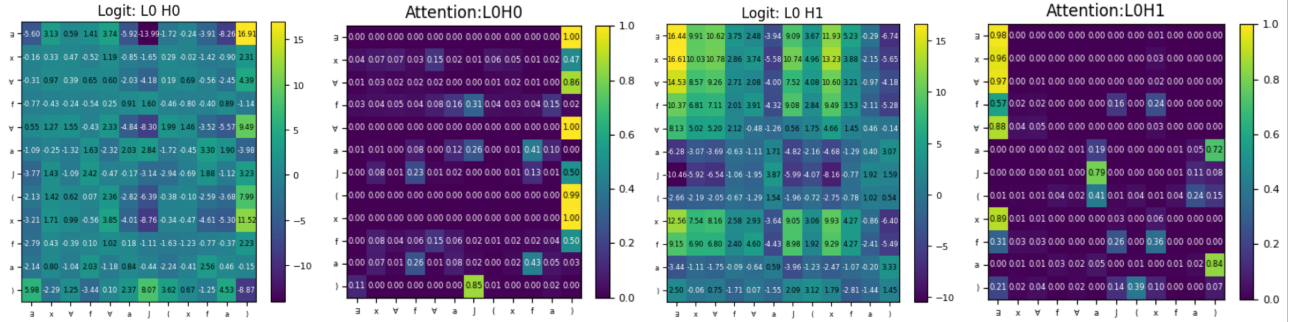
**Figure 6.** An example of value vector visualisations of a 2L2H Transformer with  $d_{model} = 8$  on *Standard* – 1 with  $QD = 3$ .

operations respectively, which can be further formalised under the saturated attention, and these operations help the model count over tokens (Merrill et al., 2021). As shown in Figure 5, the score logits in Layer0 span roughly from -300 to +300, while the logit range of Layer1 is much flatter  $[-1, 4]$  and becomes near-uniform attention after softmax. The attention weights in Layer0 are almost binary, with a value of 0 or 1.

A similar pattern appears in the value vector visualisations; see an example in Figure 6. To further prove the trend, we compute the per-head average distance among a sample’s



**Figure 7.** Per-head macro-average distance of value vectors across five FOL types.



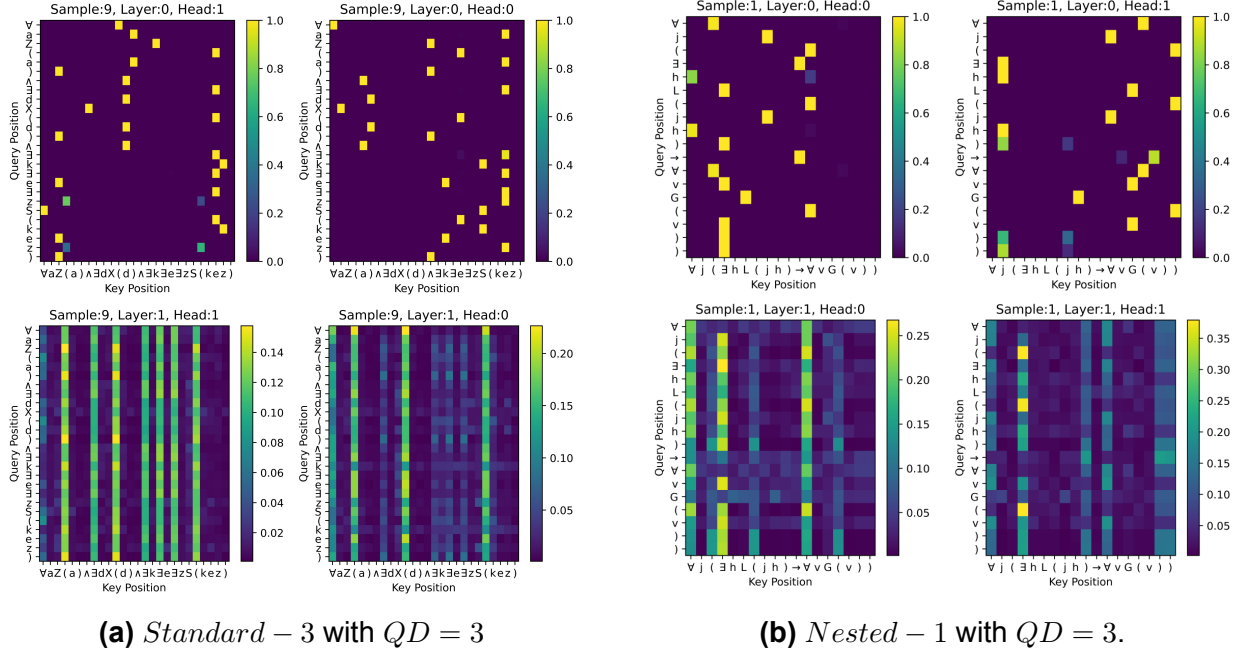
**Figure 8.** A heatmap comparison between score logits and attention weights in Layer0 of a 2L2H Transformer with  $d_{model} = 8$  on *Standard - 1*.

value vectors and report the macro-averages across samples. As shown in Figure 7, regardless of the FOL type, the distances of Layer0 heads are approximately three to five times greater than those of Layer1 heads. It quantitatively confirms the above observation that Layer0 remains a wide span, while Layer1 narrows its value range.<sup>18</sup> Beyond the overall trend, we now turn to investigating the specific responsibilities of these two layers.

**Layer0 acts as an “extractor”.** It provides the first round of interactions across positions. The sharp difference among attention weights helps the model quickly lock in the similarities and differences among tokens. In most cases, Layer0 shares a similar pattern to a single-layer Transformer. It only performs the initial pairwise matching of token relations and may make more mistakes in the complex formulas.

Previous studies (Clark et al., 2019; Michel et al., 2019; Voita et al., 2019) report the findings of characterising attention heads in larger architectures, such as BERT. Interestingly, we also observe that two attention heads play structural and relational roles respectively in Layer0, when processing *Standard - 1* with  $d_{model} = 8$ . See an example in Figure 8. From the score logits, we find that Head0 assigns negative scores among quantifiers while giving a higher value to “(” or “)” as the signal channel for structure. By contrast, Head1 prioritises the first quantifier with the highest scores from itself and other quantifiers, and excludes the selected parentheses. This reflects a mechanism similar to the tree structure: the outermost quantifier serves as the root node and gathers the

<sup>18</sup>The related heatmaps of value vectors also reveal a position-locked pattern. Across different samples, certain indices present identical value vectors, suggesting a strong positional imprint. It needs to be verified by probing the learned positional encodings.



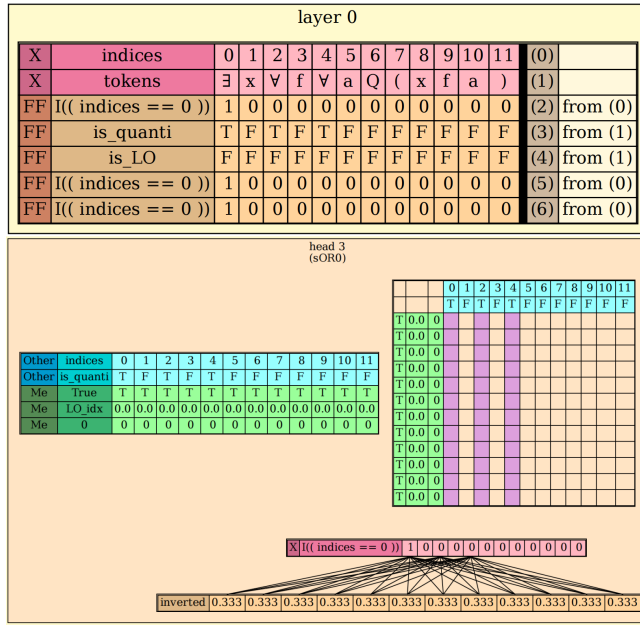
**Figure 9.** Two examples of attention heatmaps of a 2L2H Transformer on two FOL types.

information from child nodes. Then, the attention weights further refine the signals. In Head0, most tokens strongly attend to the position of the opening or closing parenthesis; In Head1, the first quantifier becomes the most popular token for attracting the task-specific information.

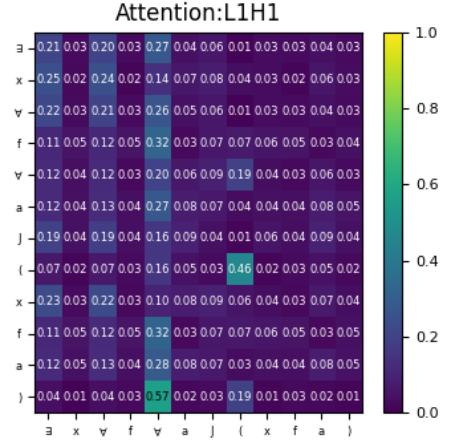
*Standard* – 1 affords the easiest setting along three features: (i) shorter sequences, (ii) fewer token-to-token interactions, and (iii) no logical operator and nesting, so the task is simplified to counting. With a smaller hidden dimension, the Transformer is forced to form more compact representations. Under these conditions, the first layer can both extract and aggregate the relevant signals, with clear head-level specialisation. One head tracks the structural boundary, while the other collects quantifiers. However, more complex patterns typically require a higher layer to complete the two steps.

**Layer1 functions as an “aggregator”.** It has two main roles: (i) integrating the important features extracted from Layer0 into target positions, and (ii) implementing a global-average behaviour via the uniform attention.

For the first role, Transformers not only aggregate structural information with token-level relations, but also learn to compose them to determine quantifier scopes. Parentheses become a key clue for the model in determining the structural hierarchy. The opening parenthesis “(” marks the end of the quantifier prefix, while the closing parenthesis “)” marks the end of the entire subformula. By focusing on these positions, the model is able to recognise the start and end of the quantifier chain. Empirically, we find that the model distinguishes quantifier scopes by aligning quantifiers with either opening or closing



(a) In RASP visualisation.



(b) In a 2L2H Transformer.

**Figure 10.** A matching head of *Standard* – 1 when processing “ $\exists x \forall f \forall a Q(xfa)$ ”.

parentheses; see two samples in Figure 9. In *Standard* types with parallel forms, the model extracts all quantifiers and simultaneously selects one parenthesis type. This step allows the whole sequence to be split into different subformulas through the specific parentheses of each subformula, and thereby all quantifiers are delimited by their corresponding scopes. In *Nested* types, the model perfectly learns the rule for computing *QD* in nested FOL formulas, which closely matches our RASP algorithm in § 4.3. One head aligns all quantifiers with opening parentheses, while another aligns them with closing parentheses. Leveraging both cues from two parenthesis types helps the model to track nesting boundaries, which allows *QD* to accumulate consistently across varying formula depth.

To our surprise, the model relies more on parentheses to recognise the structure. Compared to identifying different logical symbols, using parentheses to determine quantifier scopes is indeed a shortcut and can reduce computations. Only through the configuration with more layers or in the cases of processing *Nested* – 2 (see Figure 27 in the Appendix C) can we observe that the attention heads start to systematically attend to logical operators. This finding suggests that when faced with more challenging structures where the evidence from parentheses is insufficient, Transformers learn to develop the functions of logical connectives; otherwise, this knowledge emerges progressively with additional computational capacity.

For the second role, the average operation behaves like a global readout, making each token look up the task-specific symbols. In our tasks, the final output only depends on the positions of quantifiers and structural symbols; accordingly, the encoder places approximately uniform attention weights over these tied positions, rather than distributing

|    |               |     |     |     |     |     |     |     |     |     |     |     |     |     |                 |
|----|---------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----------------|
| X  | inverted      | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | (0)             |
| X  | valat0        | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | (1)             |
| X  | inverted      | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | (2)             |
| X  | valat0        | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | (3)             |
| FF | except0       | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | (4) from (0)    |
| FF | left_QD       | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | (5) from (1, 4) |
| FF | except0       | 1.0 | 1.0 | 1.0 | 1.0 | 1.0 | 1.0 | 1.0 | 1.0 | 1.0 | 1.0 | 1.0 | 1.0 | 1.0 | (6) from (2)    |
| FF | right_QD      | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | (7) from (6, 3) |
| FF | QD_type2_fn() | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | 1   | (8) from (7, 5) |

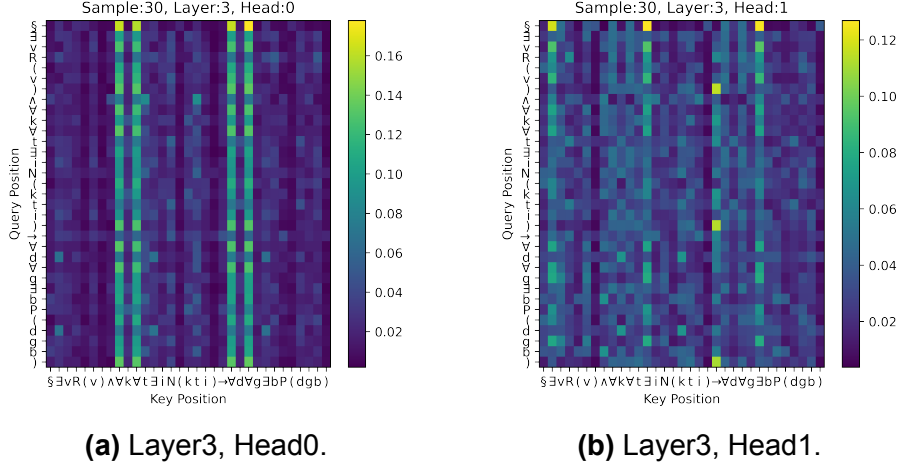
**Figure 11.** The component that simulates FFN sublayers in the RASP visualisation when processing “ $\forall gE(g) \rightarrow \forall bF(b)$ ”.

weight across all tokens. As shown in Figure 10, the attention pattern of Layer1, Head1 (the sample in Figure 5) shows that each token attends to three quantifiers with an approximately equal weight of around 0.3, which strongly aligns with the visualisation of our algorithm. Under the saturated attention in RASP, three quantifiers are attended equally, and each of them can get the attention weight  $\approx 1/3$  (0.333) from all query positions. After adjusting the range of the colour axis, this trend becomes more obvious with an even focus on the positions of all quantifiers and parentheses in Figures 9a and 9b. This global mixing enables the model to learn long-range dependencies among the whole sequence, which especially benefits the forms with longer sequences and nested structures.

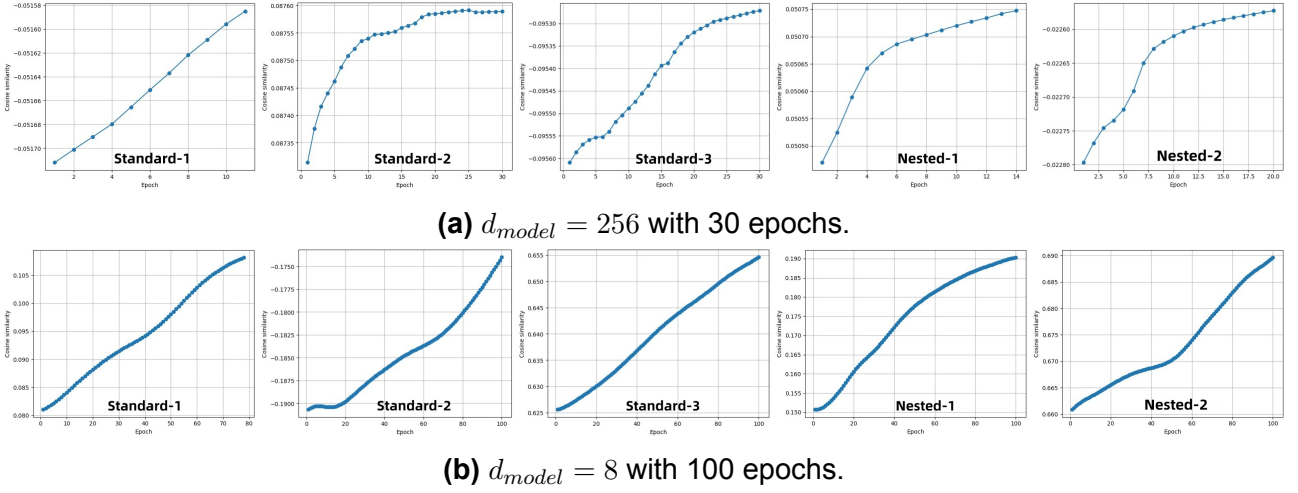
Similarly, previous studies report that in both small-sized and larger Transformer-based architectures, lower layers show position-specific attention patterns, while higher layers tend toward near-uniform weights or virtually hard attention (Abnar and Zuidema, 2020; Ebrahimi et al., 2020; Yao et al., 2021). In Bhattamishra et al.’s (2020) construction, setting the Query and Value to the identity matrices leads to a uniform attention score  $\frac{1}{i} \sum_{j=1}^i v_j$  at position  $i$  as the mean of all prefix tokens. This is appropriate for their setup on Dyck languages, where every token in the sequence contributes equally to the final result. By contrast, computing  $QD$  in our tasks requires the model to ignore predicates and variables, which means that there is no need to focus on all preceding tokens, but it should pay near-uniform attention only to the relevant quantifiers and structural symbols.

Following our design in RASP, the encoder then maps the outputs from self-attention to feed-forward networks. In Figure 11, we show an example of *Standard* – 2, which relates the process of computing  $QD$  on each side and the comparison operation to “FF” (representing FFN). Empirically, Geva et al. (2021) show that feed-forward networks implement unnormalised key-value memories. The first projection provides keys that score the input to produce coefficients, which weight the value vectors from the second projection to form the output. Under the joint effect of the output projection  $W_O$  and FFN, the results of self-attention become linearly decodable for better decision-making in later processes. Moreover, most information is stored and transmitted through the residual connection, while the remaining components work as refinements.

Overall, two-layer Transformers effectively operate as “extractor→aggregator”. The first



**Figure 12.** Two heads in Layer3 of a 4L2H Transformer with BOS-token pooling on *Standard* – 3 with  $QD = 3$ .

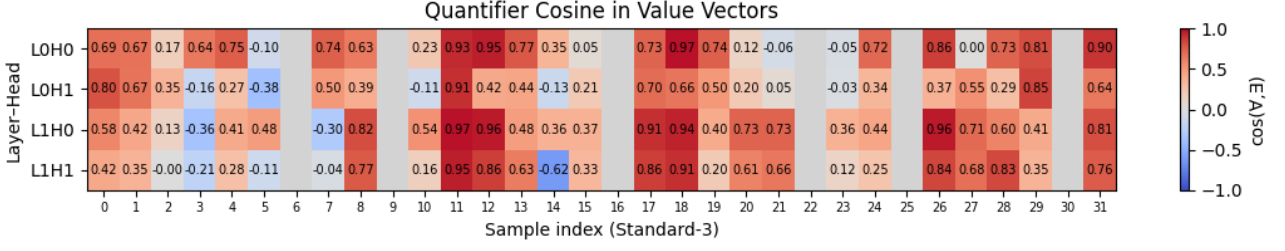


**Figure 13.** The trend of quantifier cosine similarities across all FOL types on a 2L2H Transformer during training.

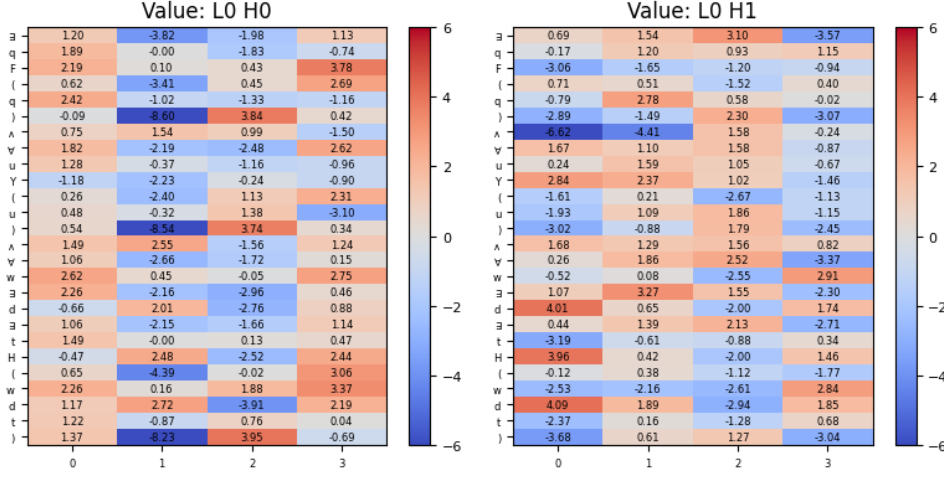
layer (Layer0) captures structural signals and token relations in a local range, while the second layer (Layer1) globally integrates information to specific tokens via near-uniform attention. The feed-forward networks and other components then map these intermediate results to the representations suitable for subsequent classification.

### 7.2.3 Relations between Quantifiers

Treating “ $\forall$ ” and “ $\exists$ ” as the same symbol is common knowledge for computing  $QD$ . Consistent with our RASP algorithms, the attention heatmaps in § 7.2.2 show that the heads broadly highlight all quantifier positions in the second layer. It suggests that the models have learned an approximate relationship between the two symbols at this stage. With a four-layer Transformer, we also observe that the attention heads in lower layers extract all quantifiers at the same time, while deeper layers tend to categorise quantifiers into two types and handle them in separate heads; see an example of a 4L2H model



**Figure 14.** Cosine values of the mean angular distance on the *Standard* – 3 validation dataset of a 2L2H Transformer with  $d_{model} = 8$ .

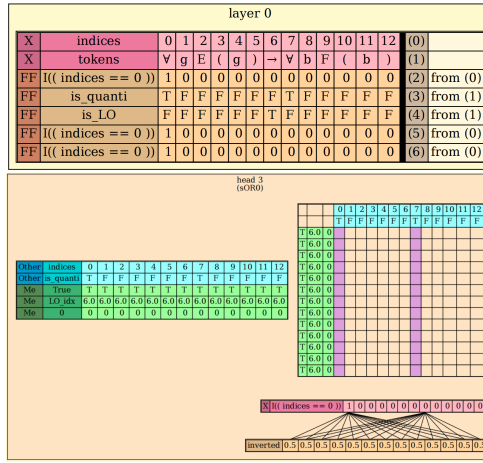


**Figure 15.** Value vectors of Sample 11 in *Standard* – 3.

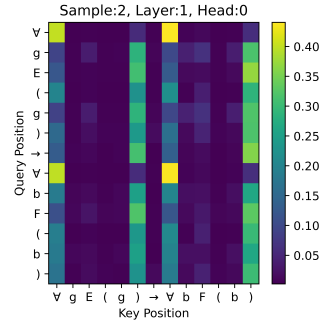
(Figure 12), where one head concentrates on all “ $\forall$ ” tokens and another head focuses on “ $\exists$ ”. This pattern indicates a progression from parallel quantifier detection to type-specific preferences as depth increases, though the task does not require such specialisation.

To investigate the above phenomenon, we further analyse the learning dynamics of the model’s quantifier representations. First, we extract the lexical embeddings for two quantifier types and track their cosine similarities throughout training. Across all FOL types and different configurations, the cosine similarities show a gradual increase over training epochs, indicating directional convergence of their representations in the embedding space. See examples (2L2H, mean pooling) in Figure 13. Second, we compute the mean angular distance between all “ $\forall$ - $\exists$ ” pairs from the value vectors, and then convert the results to cosine values. The related visualisations are extracted on the first 32 samples of each Validation set, excluding those with only one quantifier type (marked with grey in the heatmaps). According to the results of *Standard* – 3 in Figure 14 and those of other FOL types in Figure 26 (Appendix C), most heads present positive cosine similarities, and half of the heads even show strongly large values (0.8-1), suggesting that two quantifier types are projected into close vectors.

Consistent with this construction, we find that contextual processing via the value projection maps two symbols to nearby points in representational space. However, the

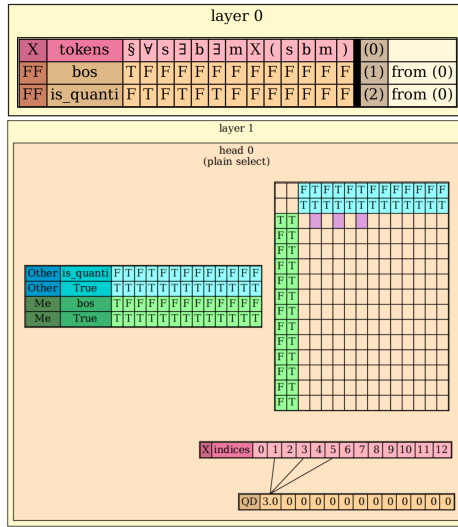


(a) In RASP visualisation.

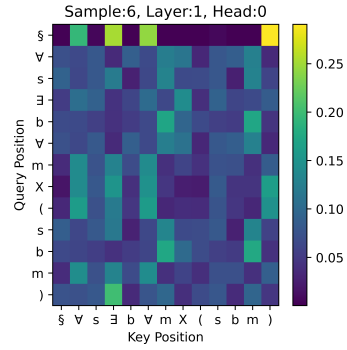


(b) In a 2L2H Transformer.

**Figure 16.** A matching head of *Standard* – 2 when processing “ $\forall gE(g) \rightarrow \forall bF(b)$ ”.



(a) In RASP visualisation.

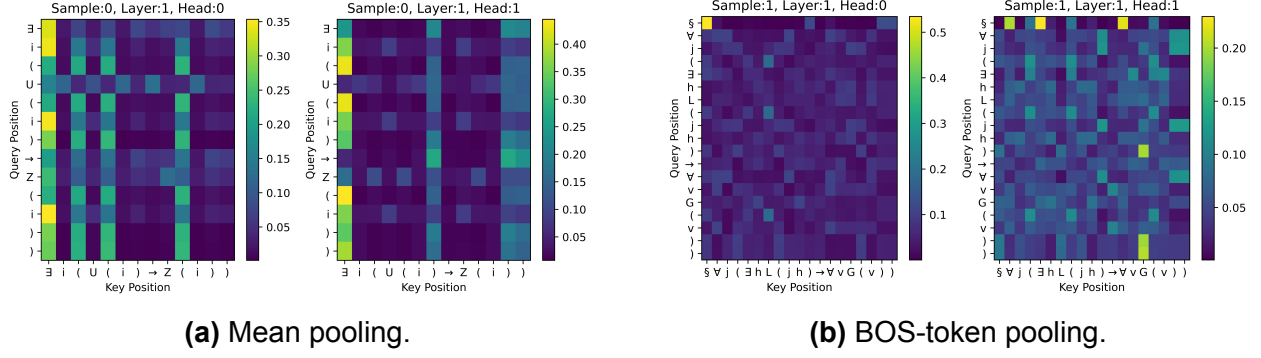


(b) In a 2L2H Transformer.

**Figure 17.** A matching head of *Standard* – 1 with BOS when processing “ $\forall s \exists b \exists m X(sbm)$ ”.

encoders retain distinctions in their actual value vectors. For example, Figure 15 shows the value vectors of Sample 11 (*Standard* – 3) in Layer1, which match the cosine similarities at 0.93 and 0.91 in Figure 14. Two quantifier types share strong directional alignment but differ in their magnitudes. These findings show that Transformers learn their relations through contextual representations rather than the base lexical embeddings, and view two symbols as functionally similar but not perfectly equal to each other. Therefore, Transformers can process all quantifiers in parallel and group them separately in higher layers.

Overall, the attention heads in early layers process all quantifiers in parallel, while deeper layers partition them by types across heads. Transformers mainly learn the relations between “ $\forall$ ” and “ $\exists$ ” through contextual value vectors, and encode them with the same functional role, but do not make them identical.



**Figure 18.** Attention heatmaps of two pooling strategies in Layer1 of a 2L2H Transformer on *Nested* – 1 with  $QD = 1$ .

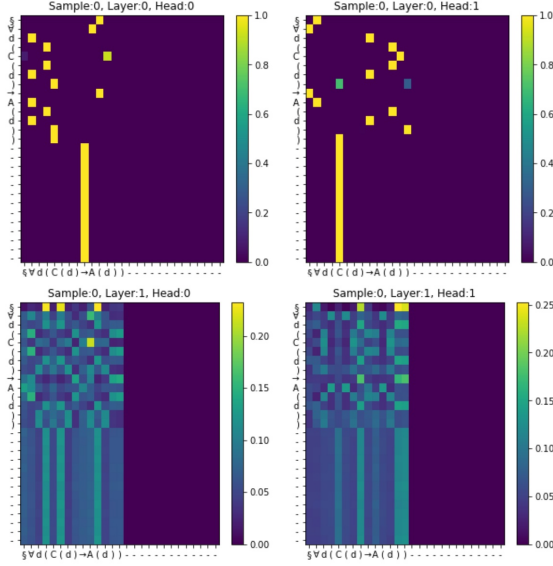
## 7.2.4 Key Information Aggregation

In experiments, we find that mean pooling outperforms BOS-token pooling<sup>19</sup> in most settings. Previous studies show that BOS tokens serve as a stack base and a neutral anchor that facilitates conditioned counting and reduces the attention heads required to handle Dyck-k languages (Ebrahimi et al., 2020; Weiss et al., 2021). However, in our tasks, the BOS-pooled models require additional heads to reach 100% scores compared to using mean pooling. Our algorithms (§ 4) also reflect that an encoder often needs more heads to implement extra aggregation into the BOS token, suggesting a small, non-pretrained Transformer must first learn the special-token anchor and route information into it. The RASP visualisations also closely mirror how the trained models select positions for gathering key information under two pooling strategies (Figures 16 and 17), except for the simultaneous extraction of parentheses. Instead, our algorithms design a stack-like scheme to segment parallel subformulas via logical operators and use parentheses to hierarchically recognise nesting levels.

From the attention heatmaps in Layer1, a mean-pooled model leverages the entire sequence to emphasise task-relevant tokens, revealing the correlations between these important tokens and the rest. As signal channels, the positions of quantifiers and parentheses realise a two-stage selection from global routing to pairwise focuses (e.g., sharing higher weights among quantifiers). Here, attention weights offer a chance for reconfirming each token’s contribution to the final output. In contrast, although Transformers can learn to use BOS for global readout, relying exclusively on it is less economical. It wastes the rich representations carried by other tokens and reduces the benefits of distributed storage.

Particularly, when evaluating on *Nested* – 1, pooling choice changes how heads allocate attention in Layer1. See Figure 18 for a direct comparison. With mean pooling, Layer1

<sup>19</sup>Mean pooling averages all tokens in the final layer, while BOS-token pooling reads the final hidden state only from the BOS token.



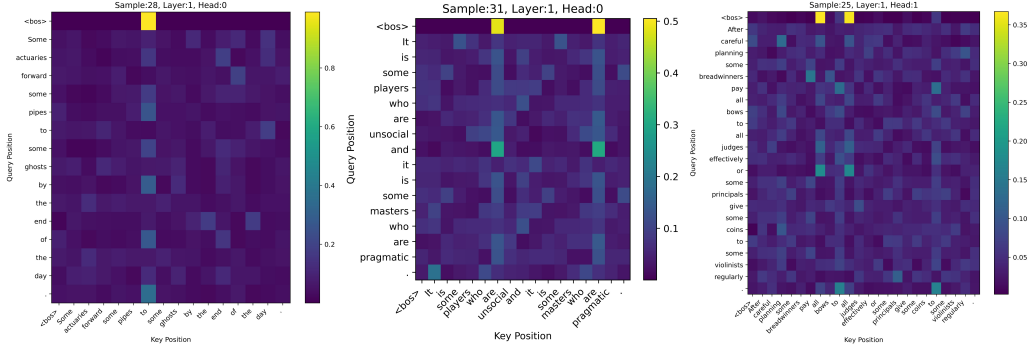
**Figure 19.** A heatmap example of a 2L2H Transformer with PAD on  $Nested - 1$  with  $QD = 1$ , where “-” represents “<pad>”.

typically uses its two heads to align quantifiers with “(” and “)” separately, reinforcing scope recognition. With BOS-token pooling, the first head is devoted to this special token for learning it as an aggregation site. Only one head remains for tracking boundaries via a single parenthesis type, which impairs the model’s ability to process quantifier scopes and nesting levels.

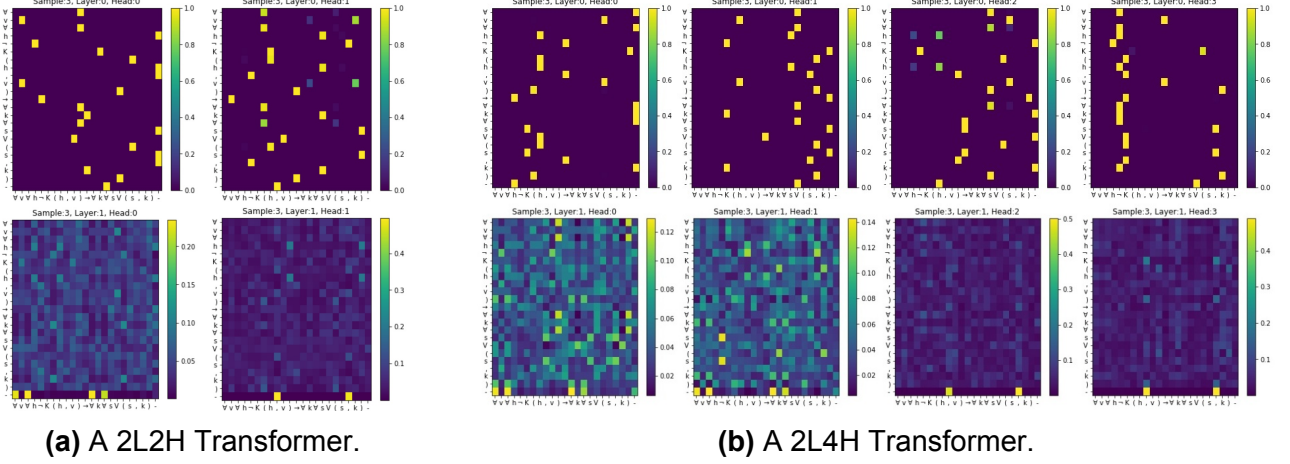
Additionally, when PAD tokens are not masked on the query side, the models regard them as storage slots for task-specific information (Figure 19). This is in accordance with the theory that transformers can store almost any amount of information on any token. Some uninformative tokens provide spare capacity that the models naturally repurpose them as auxiliary memory.

### 7.2.5 Other Patterns

First, Transformers tend to summarise the simplest rule from statistical co-occurrence. In our settings, if the commas are not removed, the models will quickly focus on “,” as a spurious cue and ignore  $QD$  recognition. They also prefer parentheses over logical operators for capturing formula structures. Moreover, this shortcut learning pattern is reflected by the extremely strong performance of BERT and the small Transformer on FOL2NS types. The related attention shows that a 2L2H model often focuses on task-irrelevant function words (e.g., “to”, “are”, “will”) rather than attending to “all” or “some” (Figure 20). This phenomenon fails to explore whether nominal participants would drive the difficulty for Transformers to recognise  $QD$  in natural sentences. It points to the fact that even with reordered syntax and inserted phrases, our generated templates may still leak stable shortcuts to the model. An interesting avenue for future work is to diversify



**Figure 20.** Three examples of a 2L2H Transformer with BOS on two FOL2NS datasets.



**Figure 21.** Same attention patterns in a 2L2H Transformer and a 2L4H Transformer.

structures beyond the current templates without changing  $QD$ , such as adding relative clauses inside the terms.

Second, redundant heads exhibit functional duplication. Across different configurations, fewer-head models display near-identical core behaviours to those with more heads. For example, Figure 21 shows overlapping attention patterns between a 2L2H model and a 2L4H model. When the number of heads substantially exceeds task demands, the models just repeat same operations in different heads. From Figure 28 in Appendix C, seven heads share a common pattern in a 5L4H Transformer. This aligns with the previous studies of Voita et al. (2019) and Michel et al. (2019) that only a few specialised heads carry most of the workload, while many others can be removed with a minor performance drop.

## 8 Discussion

Despite the interference from predicates and variables, Transformers reliably recognise quantifiers and structural tokens, and assign near-uniform attention over these positions, indicating their robustness in tracking structure under lexical distraction. Notably, certain

attention patterns are closely related to the visualisations of our algorithms. However, the alignment weakens for more complex structures (See Figures 22, 23, 24 and 25 in the Appendix C). Additionally, the upper bounds provided by RASP do not tightly match the number of layers and heads actually required in trained models to compute  $QD$ .

Several factors may explain the mismatch between our RASP algorithms and the actual Transformer behaviours. First, RASP uses restricted attention, ensuring that each head attends to designated positions and executes task-specific operations deterministically. Moreover, RASP-based visualisations may be unstable as proxies for the self-attention block, even on comparatively simple tasks (e.g., sorting, Dyck-2) (Weiss et al., 2021). By contrast, our trained models use soft attention in a more complex setting with predicate/variable interference. The first layer exhibits more diffuse patterns for pairing local relations, and all matched patterns emerge in the second layer, where task-specific information is aggregated. Our algorithms also explicitly include a second pass in the self-attention block that recombines  $QD$  evidence across subformulas. Transformers typically defer this aggregation to subsequent components, effectively reducing the attention depth required relative to the RASP programs.

Second, our algorithms are based on positions that each head counts quantifiers in a different scope, while Transformers tend to extract all quantifiers and structural symbols in parallel and jointly delimit their scope, which reduces the number of heads needed. Interestingly, although the models can learn the relation between  $\forall$  and  $\exists$ , the higher layers show different preferences that process two quantifier types separately, whereas our algorithms treat them as interchangeable tokens.

Third, our algorithms can apply to FOL formulas with arbitrary quantifier nesting, but real models cannot process unbounded sequences. In experiments, we control the maximum  $QD$  at 3, which simplifies the task and allows a smaller configuration to suffice. We also find that training time increases with higher  $QD$ ; accordingly, greater quantifier nesting will likely require additional layers and heads, bringing the model closer to the depth and width suggested by the related RASP construction.

Therefore, our future work will introduce more constrained attention mechanisms to better align the model computation with our algorithms, such as the hard attention in Yao et al.’s (2021) work for studying Dyck-k languages. Additionally, given the performance drop on longer sequences in processing formal languages (Bhattamishra et al., 2020), it is necessary to further evaluate Transformers’ generalisation abilities to extended inputs and more complex structures.

## 9 Conclusion

We theoretically and experimentally demonstrate that Transformers can recognise quantifier depth in FOL formulas with increasing structural complexity. Our findings show a productive framework on FOL for studying the interplay between performance and interpretability of the self-attention block. It may also shed light on the success of Transformers in modelling long-distance, hierarchical dependencies in natural languages.

## Acknowledgments

I am sincerely grateful to my supervisor for his guidance, encouragement, support, and constructive feedback throughout this project. I also thank the academic staff of the Departments of Linguistics and Computer Science for their dedicated teaching and valuable suggestions over the past year.

## References

- Gail Weiss, Yoav Goldberg, and Eran Yahav (2021). ‘Thinking Like Transformers’. In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. PMLR, pp. 11080–11090.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin (2017). ‘Attention is all you need’. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*, pp. 6000–6010.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2019). ‘BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding’. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, pp. 4171–4186.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv: 1907.11692.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu (2023). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. arXiv: 1910.10683.

- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio (2016). *Neural Machine Translation by Jointly Learning to Align and Translate*. arXiv: 1409.0473.
- Satwik Bhattamishra, Kabir Ahuja, and Navin Goyal (2020). ‘On the Ability and Limitations of Transformers to Recognize Formal Languages’. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, pp. 7096–7116.
- Javid Ebrahimi, Dhruv Gelda, and Wei Zhang (2020). ‘How Can Self-Attention Networks Recognize Dyck-n Languages?’ In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, pp. 4301–4306.
- Shunyu Yao, Binghui Peng, Christos Papadimitriou, and Karthik Narasimhan (2021). ‘Self-Attention Networks Can Process Bounded Hierarchical Languages’. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, pp. 3770–3785.
- N. Chomsky (1956). ‘Three models for the description of language’. In: *IRE Transactions on Information Theory* 2.3, pp. 113–124.
- Fred Karlsson (2007). ‘Constraints on Multiple Center-Embedding of Clauses’. In: *Journal of Linguistics* 43.2, pp. 365–392.
- Lifeng Jin, Finale Doshi-Velez, Timothy Miller, William Schuler, and Lane Schwartz (2018). ‘Depth-bounding is effective: Improvements and evaluation of unsupervised PCFG induction’. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 2721–2731.
- Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning (2019). ‘What Does BERT Look at? An Analysis of BERT’s Attention’. In: *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. Association for Computational Linguistics, pp. 276–286.
- Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov (2019). ‘Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned’. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 5797–5808.
- Samira Abnar and Willem Zuidema (2020). ‘Quantifying Attention Flow in Transformers’. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 4190–4197.

- Peter Griinwald (1996). 'A Recurrent Network That Performs a Context-Sensitive Prediction Task'. In: *Proceedings of the Eighteenth Annual Conference of The Cognitive Science Society*. Lawrence Erlbaum, pp. 18–335.
- Gail Weiss, Yoav Goldberg, and Eran Yahav (2018). 'On the Practical Computational Power of Finite Precision RNNs for Language Recognition'. In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, pp. 740–745.
- Mirac Suzgun, Yonatan Belinkov, Stuart Shieber, and Sebastian Gehrmann (2019). 'LSTM Networks Can Perform Dynamic Counting'. In: *Proceedings of the Workshop on Deep Learning and Formal Languages: Building Bridges*. Association for Computational Linguistics, pp. 44–54.
- William Merrill, Vivek Ramanujan, Yoav Goldberg, Roy Schwartz, and Noah A. Smith (2021). 'Effects of Parameter Norm Growth During Transformer Training: Inductive Bias from Gradient Descent'. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 1766–1781.
- Paul Michel, Omer Levy, and Graham Neubig (2019). 'Are Sixteen Heads Really Better than One?' In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc.
- Dmitry Nikolaev and Sebastian Padó (2023). 'Representation biases in sentence transformers'. In: *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 3701–3716.
- Jidong Tian, Yitian Li, Wenqing Chen, Liqiang Xiao, Hao He, and Yaohui Jin (2021). 'Diagnosing the First-Order Logical Reasoning Ability Through LogicNLI'. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 3738–3747.
- Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alex Wardle-Solano, Hannah Szabo, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Alexander R. Fabbri, Wojciech Kryscinski, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying, Arman Cohan, and Dragomir Radev (2024). *FOLIO: Natural Language Reasoning with First-Order Logic*. arXiv: 2209.00840.

- Abhinav Lalwani, Tasha Kim, Lovish Chopra, Christopher Hahn, Zhijing Jin, and Mrinmaya Sachan (2025). *Autoformalizing Natural Language to First-Order Logic: A Case Study in Logical Fallacy Detection*. arXiv: 2405.02318.
- L. T. F. Gamut (1990). *Logic, Language, and Meaning. Introduction to Logic*. Vol. 1. University of Chicago Press.
- Elizabeth Coppock and Lucas Champollion (2025). *Invitation to Formal Semantics. Draft textbook*. University of Oxford; New York University.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun (2016). ‘Deep Residual Learning for Image Recognition’. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.
- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton (2016). *Layer Normalization*. arXiv: 1607.06450.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun (2015). ‘Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification’. In: *Proceedings of the 2015 IEEE International Conference on Computer Vision (ICCV)*, pp. 1026–1034.
- Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy (2021). ‘Transformer Feed-Forward Layers Are Key-Value Memories’. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 5484–5495.

## Appendices

### A FOL2NS Template

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pattern 1: All/Some <math>\{UP_1\}</math> are <math>\{UP_2\}</math>.</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| $\forall x(P(x) \rightarrow Q(x))$<br>$\exists x(P(x) \wedge Q(x))$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Pattern 2: All/Some <math>\{UP_1\}</math> <math>\{BP_1\}</math> all/some <math>\{UP_2\}</math>.</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| $\forall x(P(x) \rightarrow \forall y(Q(y) \rightarrow R(x, y)))$<br>$\forall x(P(x) \rightarrow \exists y(Q(y) \wedge R(x, y)))$<br>$\exists x(P(x) \wedge \exists y(Q(y) \wedge R(x, y)))$<br>$\exists x(P(x) \wedge \forall y(Q(y) \rightarrow R(x, y)))$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Pattern 3: All/Some <math>\{UP_1\}</math> <math>\{TP_1\}</math> all/some <math>\{UP_2\}</math> to all/some <math>\{UP_2\}</math>.</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| $\forall x(P(x) \rightarrow \forall(Q(y) \rightarrow \forall z(S(z) \rightarrow R(x, y, z))))$<br>$\forall x(P(x) \rightarrow \exists(Q(y) \wedge \forall z(S(z) \rightarrow R(x, y, z))))$<br>$\forall x(P(x) \rightarrow \forall(Q(y) \rightarrow \exists z(S(z) \wedge R(x, y, z))))$<br>$\forall x(P(x) \rightarrow \exists(Q(y) \wedge \exists z(S(z) \wedge R(x, y, z))))$<br>$\exists x(P(x) \wedge \exists(Q(y) \wedge \exists z(S(z) \wedge R(x, y, z))))$<br>$\exists x(P(x) \wedge \forall(Q(y) \rightarrow \exists z(S(z) \wedge R(x, y, z))))$<br>$\exists x(P(x) \wedge \exists(Q(y) \wedge \forall z(S(z) \rightarrow R(x, y, z))))$<br>$\exists x(P(x) \wedge \forall(Q(y) \rightarrow \forall z(S(z) \rightarrow R(x, y, z))))$ |

**Table 8.** Three patterns and their related English templates in *Nested* – 2.

## B Selection of Predicates and Entities

### B.1 Unary Predicate: 98 words

Active, Alert, Ambitious, Artistic, Bored, Brave, Busy, Calm, Careless, Cautious, Charming, Cheerful, Clever, Clumsy, Cold, Confident, Creative, Critical, Curious, Demanding, Determined, Diligent, Disorganized, Distracted, Efficient, Elegant, Energetic, Experienced, Fair, Fearless, Focused, Friendly, Funny, Generous, Graceful, Hardworking, Helpful, Honest, Humble, Idealistic, Impatient, Junior, Kind, Late, Lazy, Loyal, Messy, Modest, Motivated, Naive, Nervous, Neutral, New, Old, Open, Organized, Passionate, Patient, Picky, Polite, Pragmatic, Proud, Punctual, Quiet, Realistic, Rebellious, Relaxed, Reliable, Reserved, Rude, Selfish, Senior, Serious, Short, Shy, Silent, Skilled, Slow, Smart, Social, Strict, Strong, Stubborn, Stylish, Talented, Talkative, Tall, Thoughtful, Tired, Unfair, Unreliable, Unsocial, Visionary, Warm, Weak, Wise, Witty, Young.

## **B.2 Binary Predicate: 88 words**

Accompany, Accuse, Admire, Advise, Align, Approach, Argue, Assist, Betray, Blame, Brief, Challenge, Collaborate, Comment, Compare, Compete, Compliment, Confront, Consult, Contact, Convince, Criticize, Deceive, Demand, Discipline, Discuss, Dismiss, Doubt, Employ, Engage, Envy, Evaluate, Fire, Follow, Fund, Greet, Guide, Hate, Help, Ignore, Inform, Instruct, Insult, Interrupt, Invite, Involve, Judge, Know, Lecture, Like, Listen, Manage, Mentor, Monitor, Motivate, Negotiate, Notify, Observe, Oppose, Pay, Persuade, Praise, Prefer, Protect, Provoke, Punish, Question, Refer, Reject, Remind, Replace, Report, Request, Respect, Reward, Schedule, Scold, Shadow, Sponsor, Supervise, Support, Teach, Train, Trust, Undermine, Love, Value, Warn.

## **B.3 Ternary Predicate: 35 words**

Allocate, Assign, Award, Bring, Contribute, Convey, Consign, Delegate, Deliver, Dispatch, Distribute, Donate, Entrust, Explain, Forward, Furnish, Give, Grant, Hand, Introduce, Lend, Loan, Offer, Pass, Pay, Post, Present, Provide, Recommend, Sell, Send, Share, Show, Supply, Transfer.

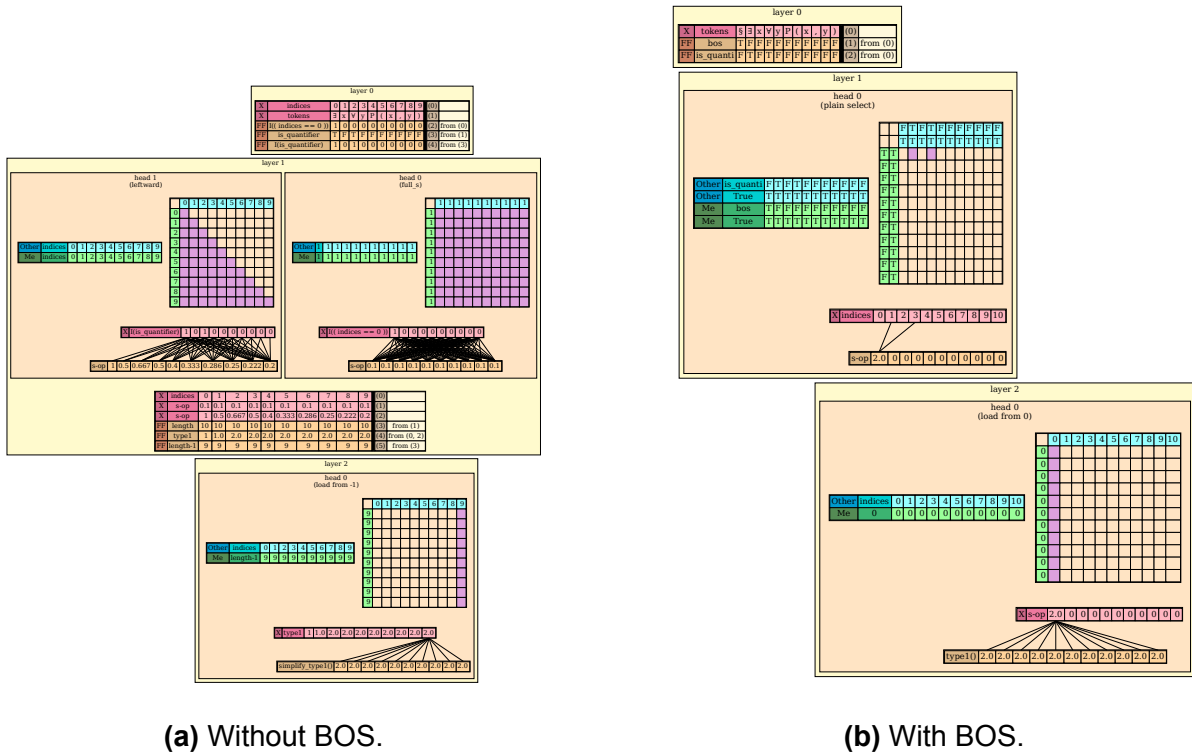
## **B.4 Person: 128 words**

Accountants, Actors, Actuaries, Adults, Advisors, Agents, Allergists, Analysts, Anthropologists, Archaeologists, Artists, Astronomers, Athletes, Attackers, Audiologists, Auditors, Babies, Bailiffs, Bakers, Ballerinas, Barbers, Bartenders, Bloggers, Boxers, Breadwinners, Butchers, Butlers, Captains, Cartographers, Cashiers, Chiropractors, Cleaners, Clerks, Conductors, Cooks, Cricketers, Crooks, Cyclists, Cynics, Dancers, Defenders, Dentists, Directors, Drillers, Drivers, Economists, Electricians, Engineers, Epidemiologists, Experts, Farmers, Fighters, Firemen, Fishermen, Footballers, Foresters, Ghosts, Grandmasters, Guests, Gymnasts, Hairdressers, Helpers, Historians, Hosts, Jewelers, Judges, Jurors, Kings, Knights, Lawyers, Lecturers, Librarians, Machinists, Masters, Mathematicians, Mechanics, Monologists, Musicians, Opticians, Painters, Parents, Patients, Pavers, Philosophers, Photographers, Physicians, Physicists, Pilots, Players, Playmakers, Plumbers, Poets, Policemen, Princes, Princesses, Principals, Prisoners, Professors, Psychologists, Publishers, Quants, Queens, Researchers, Roofers, Sailors, Scholars, Scientists, Scorers, Scribes, Secretaries, Settlers, Sheriffs, Soldiers, Strategists, Students, Surgeons, Surveyors, Teachers, Technicians, Therapists, Tourists, Traders, Veterinarians, Violinists, Visitors, Waiters, Warlords, Witches.

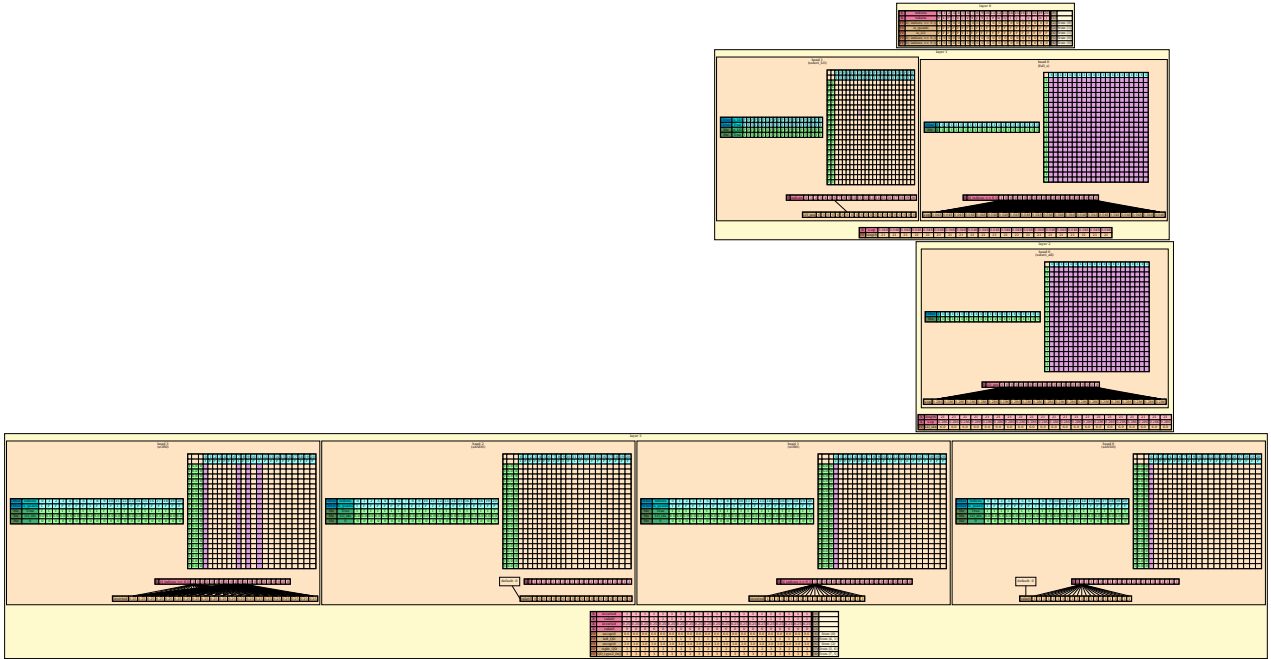
## B.5 Thing: 88 words

Apples, Amulets, Arrows, Axes, Backpacks, Batteries, Belts, Bells, Boots, Bolts, Books, Boxes, Bowls, Bows, Bracelets, Bracers, Brooches, Buckets, Candles, Chalices, Chests, Cogs, Coins, Compasses, Crates, Crossbows, Crowbars, Cups, Daggers, Drums, FishingRods, Flasks, Flutes, Gauntlets, Gems, Glasses, Gloves, Greaves, Hammers, Hats, Helmets, Horns, Jars, Keys, Lanterns, Locket, Maps, Masks, Mirrors, Nets, Necklaces, Notebooks, OilFlasks, Orbs, Paintings, Pauldrons, Plates, Pliers, Pears, Pipes, Pouches, Potions, Quills, Ropes, Runes, Sashes, Satchels, ScrollCases, Scrolls, Saws, Screwdrivers, Shields, Shovels, Spears, Staffs, Statues, SwordSheaths, Swords, Tablets, Talismans, Tongs, Torches, Trinkets, Trunks, Vases, Vials, Wands, Wrenches.

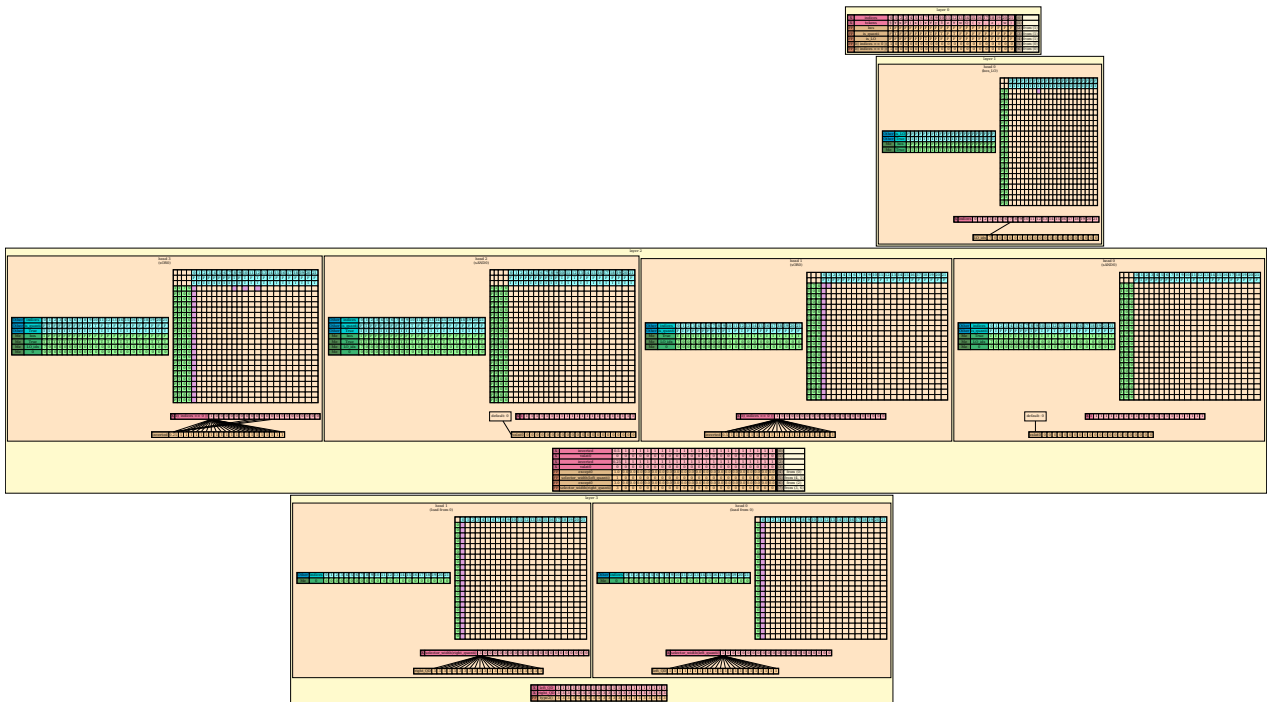
## C Plots



**Figure 22.** An example of the full RASP visualisation on *Standard* – 1 with  $QD = 2$ .



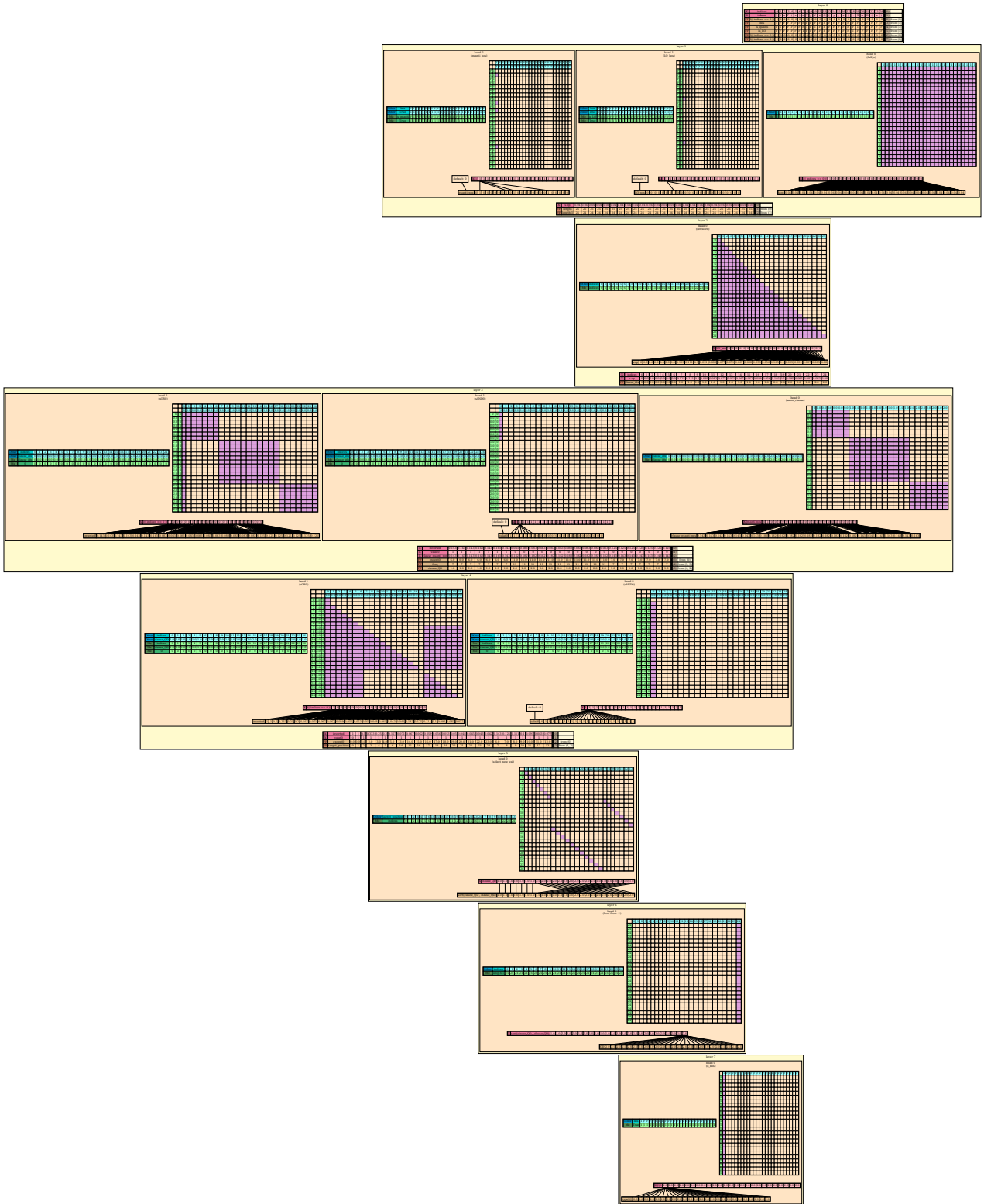
(a) Without BOS.



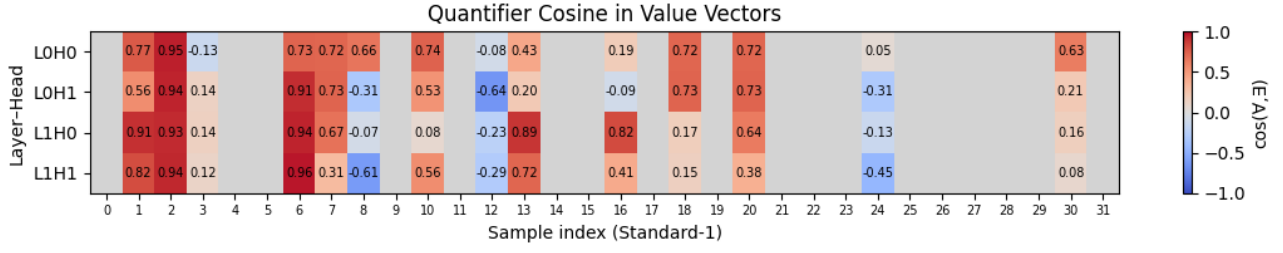
(b) With BOS.

**Figure 23.** An example of the full RASP visualisation on *Standard – 2* with  $QD = 3$ .

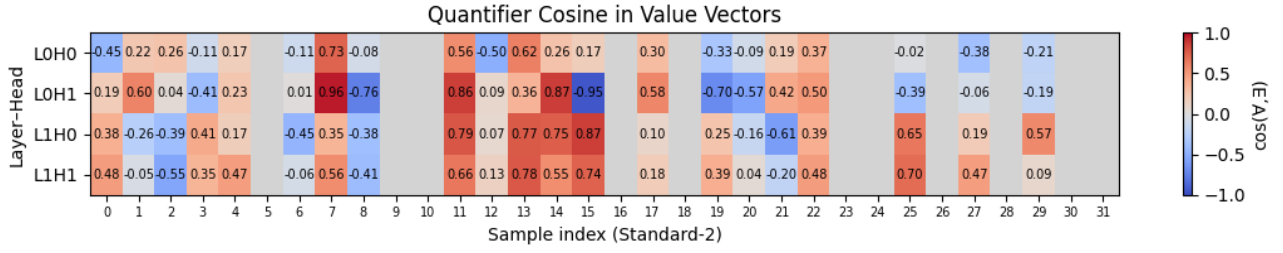




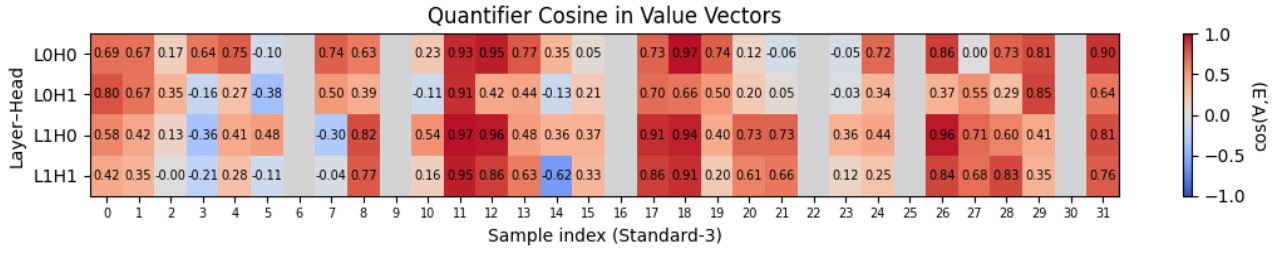
**Figure 25.** An example of the full RASP visualisation with BOS on *Standard - 3* with  $QD = 2$ .



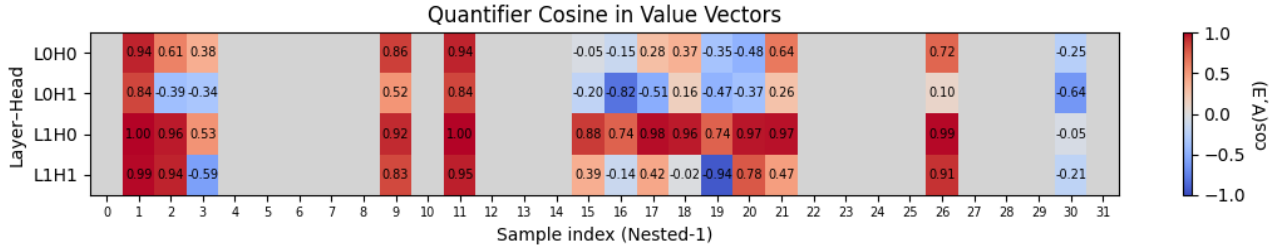
(a) *Standard* – 1.



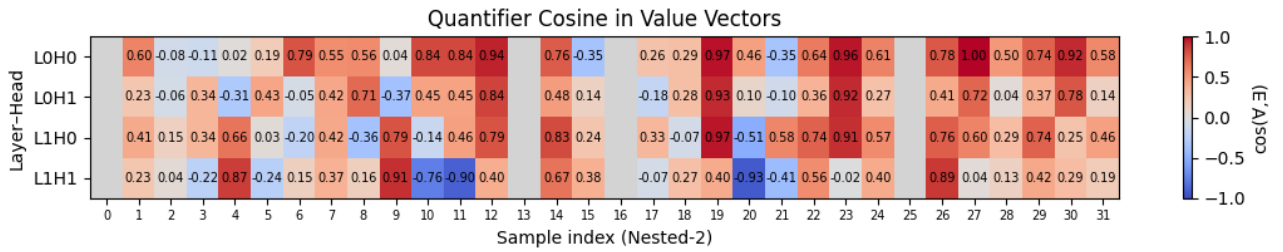
(b) *Standard* – 2.



(c) *Standard* – 3.

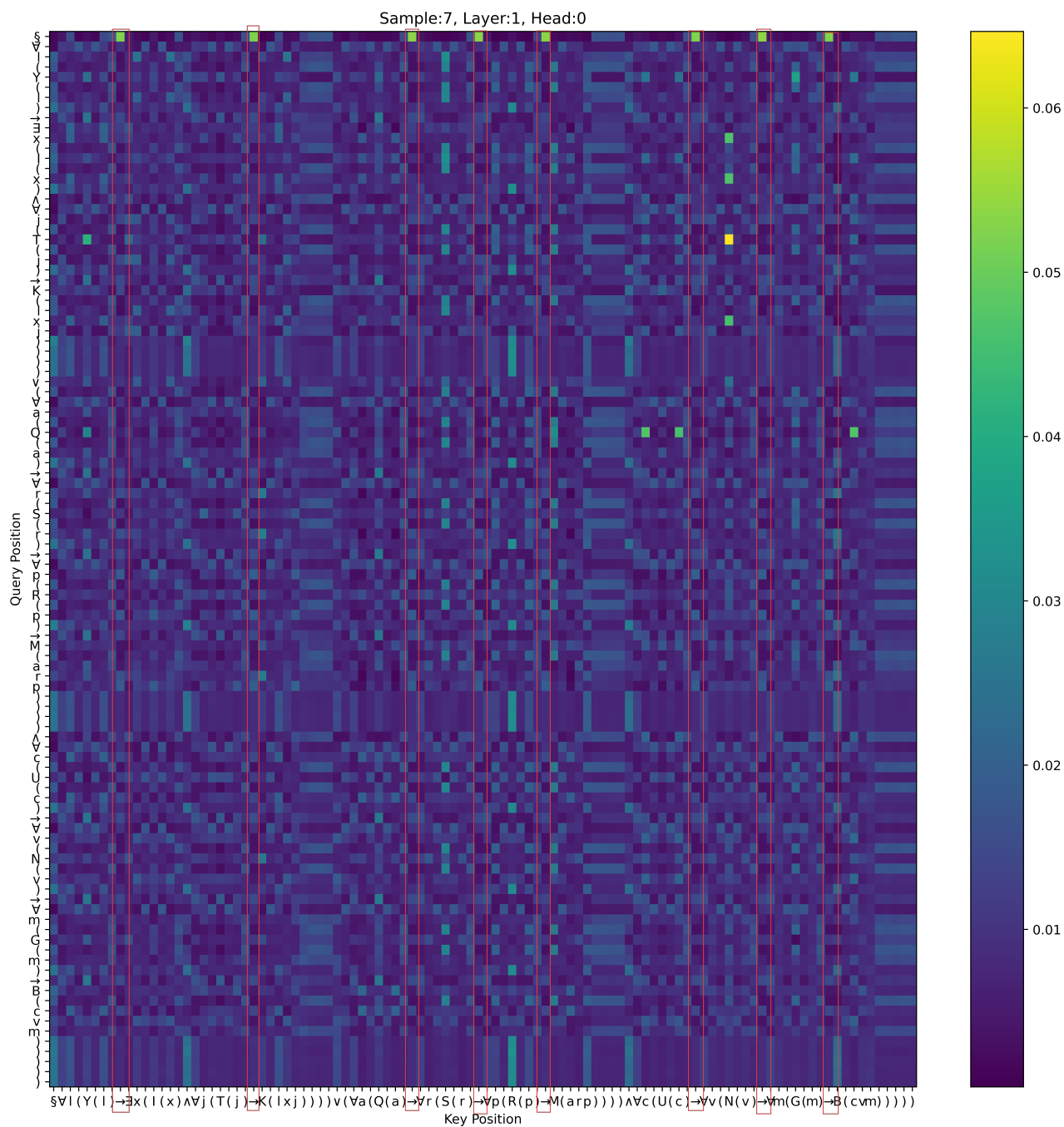


(d) *Nested* – 1.

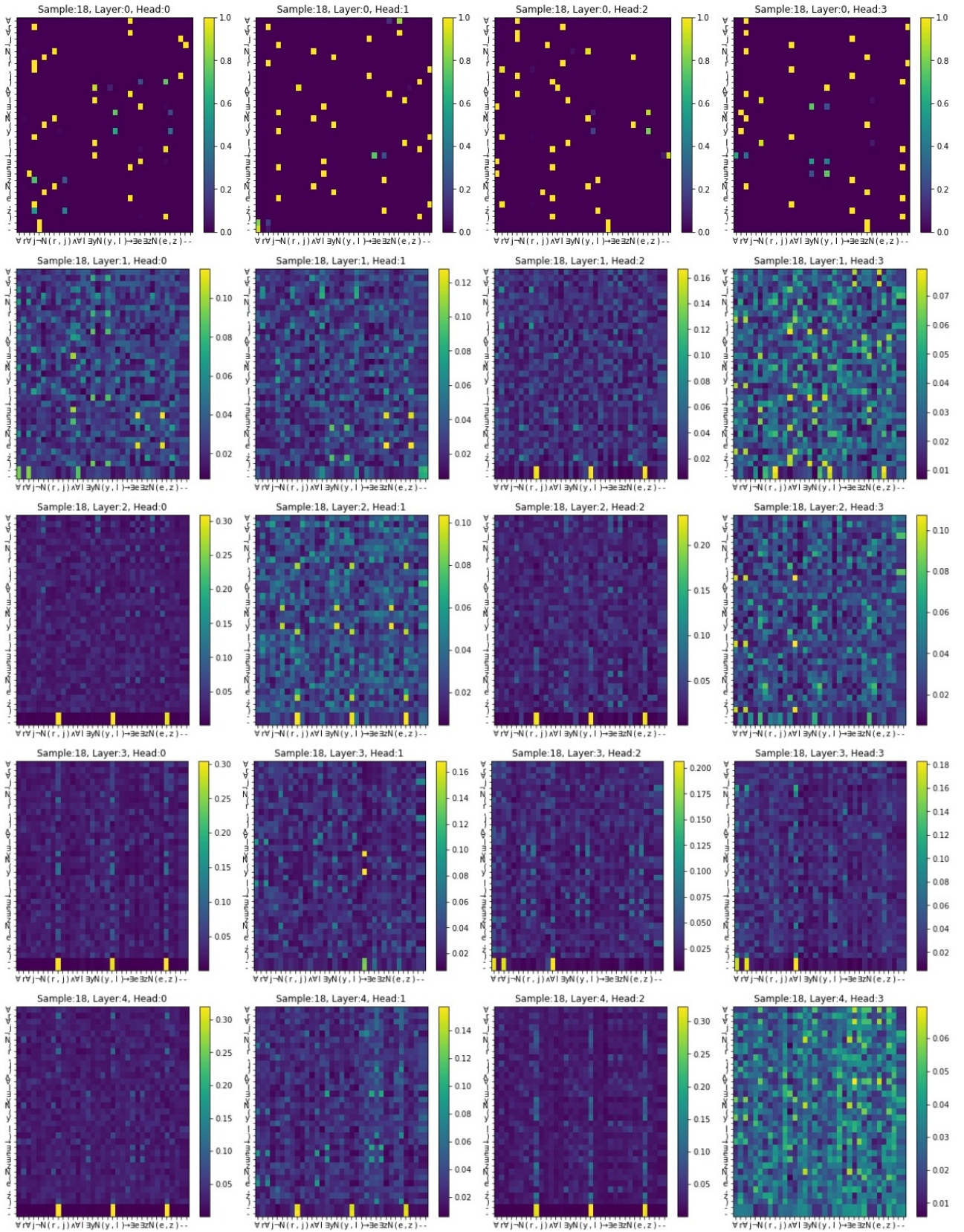


(e) *Nested* – 2.

**Figure 26.** Cosine values of the mean angular distance on five validation datasets of a 2L2H Transformer with  $d_{model} = 8$ .



**Figure 27.** An examples of focusing on logical operators in L1H0 of a 5L4H Transformer.



**Figure 28.** An examples of repeated behaviours in a 5L4H Transformer (L1H2, L2H0, L2H2, L3H0, L4H0, L4H1, and L4H2).